MỤC LỤC

MỤC LỤC i
DANH MỤC HÌNH ẢNH iv
DANH MỤC BẢNG BIỂU v
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT vi
LỜI CẢM ƠN 1
LỜI MỞ ĐẦU 2
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI 3
1.1. Giới thiệu đề tài 3
1.2. Tính cấp thiết của đề tài 3
1.3. Mục tiêu nghiên cứu 4
1.3.1. Mục tiêu tổng quát 4
1.3.2. Mục tiêu kỹ thuật cụ thể 5
1.4. Nội dung nghiên cứu và phương pháp tiếp cận 5
1.4.1. Nội dung nghiên cứu 5
1.4.2. Phương pháp tiếp cận 6
1.5. Phạm vi và đối tượng nghiên cứu 6
1.6. Bố cục của báo cáo đồ án tốt nghiệp 7
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 8
2.1. Tổng quan về PostgreSQL và Redis 8
2.1.1. PostgreSQL là gì? 8
2.1.2. Extension pgvector và chỉ mục HNSW 8
2.1.3. Redis là gì? 9
2.2. Tổng quan về Hexagonal Architecture 10
2.2.1. Hexagonal Architecture là gì? 10
2.2.2. Kiến trúc Ports and Adapters 11
2.3. Tổng quan về Advanced RAG 12
2.3.1. RAG là gì? (Retrieval-Augmented Generation) 12
2.3.2. Advanced RAG là gì? 13
2.3.3. Hybrid Search và thuật toán RRF 14
2.4. Tổng quan về Knowledge Graph và Graph Wiki 15
2.4.1. Knowledge Graph là gì? (Đồ thị tri thức) 15

2.4.2. GraphRAG là gì? 16
2.4.3. Graph Wiki là gì? (Compiled Academic Wiki) 17
2.5. Tìm hiểu về Agentic AI và ReAct 17
2.5.1. Agentic AI là gì? 17
2.5.2. ReAct Pattern là gì? (Reasoning and Acting) 19
2.5.3. Cơ chế Reflexion 19
2.6. Tổng quan về LangGraph 20
2.6.1. LangGraph là gì? 20
2.6.2. Kiến trúc StateGraph điều phối Agent 21
2.7. Tìm hiểu về Azure OpenAI và E2B Sandbox 21
2.7.1. Azure OpenAI Service là gì? 21
2.7.2. E2B Sandbox là gì? 22
2.7.3. Mô hình bảo mật BYOK (Bring Your Own Key) 23
2.8. Tổng quan về Langfuse 23
2.8.1. Langfuse là gì? (LLM Tracing & Observability) 23
2.8.2. Cơ chế theo dõi chi phí và độ trễ LLM 24
2.9. Các công cụ và ngôn ngữ phát triển hệ thống 25
2.9.1. Python và FastAPI 25
2.9.2. TypeScript và React 25
2.9.3. Docker, Pytest và Vitest 26
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG 27
3.1. Khảo sát quy trình nghiệp vụ và mô hình hóa bài toán thực tế 27
3.1.1. Bối cảnh nghiên cứu khoa học tại VinUniversity 27
3.1.2. Phân tích quy trình nghiên cứu truyền thống (Luồng AS-IS) 27
3.1.3. Mô hình quy trình nghiên cứu đề xuất (Luồng TO-BE) 28
3.2. Phân rã năng lực chức năng và Đặc tả ca sử dụng cốt lõi 29
3.2.1. Xác định các tác nhân và vai trò tương tác 29
3.2.2. Mô hình phân rã khối năng lực hệ thống toàn trình 29
3.2.3. Đặc tả chi tiết các ca sử dụng cốt lõi 30
3.3. Thiết kế kiến trúc suy luận và Vết thực thi tác nhân 30
3.3.1. Kiến trúc tự phản biện hai dải màu và Vòng lặp phản hồi đóng .. 30
3.3.2. Trace thực thi tác nhân tương tác đa bước (Agent Execution Trace) 34
3.4. Thiết kế cơ sở dữ liệu và Lược đồ thực thể tri thức 36

3.4.1. Lược đồ thực thể tri thức và Dòng chảy dữ liệu học thuật 36
3.4.2. Từ điển dữ liệu cho các thực thể cơ sở dữ liệu cốt lõi 36
CHƯƠNG 4: CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM 39
4.1. Môi trường cài đặt và Kiến trúc Monorepo 39
4.1.1. Cấu hình môi trường phần cứng và phần mềm 39
4.1.2. Kiến trúc kho mã nguồn Monorepo và rào chắn kiểm thử ranh giới AST 40
4.2. Giao diện các phân hệ chính của VeriScholar 41
4.2.1. Phân hệ Khám phá tài liệu khoa học 43
4.2.2. Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box 45
4.2.3. Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả 46
4.2.4. Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX 47
4.3. Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh 48
4.3.1. Quy trình kiểm thử 4 Cổng bắt buộc 48
4.3.2. Kết quả kiểm tra độ tin cậy 49
4.4. Số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud 49
4.4.1. Độ trễ phản hồi hệ thống và thời gian đến token đầu tiên 49
4.4.2. Độ chính xác định vị tọa độ và tỷ lệ tự sửa lỗi LaTeX 49
4.4.3. Nhật ký kiểm chứng thực tế trên Langfuse Cloud 49
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 51
TÀI LIỆU THAM KHẢO 52

DANH MỤC HÌNH ẢNH

Hình 2-1: Mô hình phân rã module và cấu trúc Cổng – Điều hợp (Ports and Adapters) 12
Hình 2-2: Quy trình Data Pipeline trích xuất tri thức phân tầng GraphRAG. 16
Hình 2-3: Kiến trúc Tác nhân AI Độc lập (LLM-Powered Autonomous Agent) 18

Hình 3-1: Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE 29
Hình 3-2: Mô hình phân rã cấu trúc năng lực của hệ thống VeriScholar..... 30
Hình 3-3: Kiến trúc Phân tầng Tự phản biện hai dải màu (Dual-Band)..... 35
Hình 3-4: Trace thực thi tác nhân tương tác đa bước (Multi-Step Agent Execution Trace) 35
Hình 3-5: Lược đồ thực thể quan hệ lai (Hybrid Relational-Vector-Graph ERD)..... 37

Hình 4-1: Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar..... 43
Hình 4-2: Giao diện Phân hệ Khám phá tài liệu khoa học 45
Hình 4-3: Giao diện Phân hệ Đọc sâu và Bounding Box..... 45
Hình 4-4: Giao diện Hồ sơ học giả và Két sắt BYOK 47
Hình 4-5: Giao diện Phân hệ Soạn thảo LaTeX 47

DANH MỤC BẢNG BIỂU

Bảng 2-1: So sánh đặc tính giữa tìm kiếm ngữ nghĩa Dense Vector và từ khóa BM25 …… 15

Bảng 3-1: Bảng đặc tả chi tiết ca sử dụng UC-01 (Khám phá bài báo học thuật hai tầng tin cậy) …… 31

Bảng 3-2: Bảng đặc tả chi tiết ca sử dụng UC-03 (Hỏi đáp Hybrid RAG có tọa độ) …… 32

Bảng 3-3: Bảng đặc tả chi tiết ca sử dụng UC-06 (Tác nhân tự sửa lỗi bản thảo LaTeX) …… 33

Bảng 3-4: Đặc tả cấu trúc từ điển dữ liệu thực thể Phân đoạn văn bản (Document Chunks) …… 37

Bảng 3-5: Đặc tả cấu trúc từ điển dữ liệu thực thể Khối tệp nhị phân CAS (Storage Blobs)…… 38

Bảng 3-6: Đặc tả cấu trúc từ điển dữ liệu thực thể Két sắt khóa cá nhân (User Secrets Vault) …… 38

Bảng 4-1: Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc …… 42

Bảng 4-2: Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh…… 49

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

STT	Ký hiệu / viết tắt	Ý nghĩa
1	ACID	Atomicity, Consistency, Isolation, Durability – Bốn thuộc tính toàn vẹn giao dịch cơ sở dữ liệu
2	AI	Artificial Intelligence – Trí tuệ nhân tạo
3	API	Application Programming Interface – Giao diện lập trình ứng dụng
4	ASGI	Asynchronous Server Gateway Interface – Giao diện cổng máy chủ bất đồng bộ Python
5	BBox	Bounding Box – Khung hộp chữ nhật định vị tọa độ
6	BM25	Best Matching 25 – Thuật toán xếp hạng tìm kiếm từ khóa toàn văn
7	BYOK	Bring Your Own Key – Cơ chế người dùng tự cung cấp khóa API cá nhân
8	CAS	Content-Addressable Storage – Hệ thống lưu trữ định danh theo nội dung băm
9	CI/CD	Continuous Integration / Continuous Deployment – Tích hợp và Triển khai liên tục
10	CTE	Common Table Expression – Bảng biểu thức chung trong truy vấn SQL đệ quy
11	DOI	Digital Object Identifier – Định danh đối tượng kỹ thuật số bài báo khoa học
12	ERD	Entity-Relationship Diagram – Sơ đồ quan hệ thực thể cơ sở dữ liệu
13	FTS	Full-Text Search – Tìm kiếm toàn văn cơ sở dữ liệu
14	GIN	Generalized Inverted Index – Chỉ mục đảo tổng quát trong PostgreSQL
15	GraphRAG	Graph Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất qua đồ thị
16	HNSW	Hierarchical Navigable Small World – Cấu trúc đồ thị chỉ mục tìm kiếm vector gần đúng
17	IoU	Intersection over Union – Chỉ số trùng khớp diện tích hộp tọa độ

STT	Ký hiệu / viết tắt	Ý nghĩa
18	JWT	JSON Web Token – Chuỗi mã định danh phiên người dùng an toàn
19	LLM	Large Language Model – Mô hình ngôn ngữ lớn
20	ORCID	Open Researcher and Contributor ID – Mã định danh học giả quốc tế 16 số
21	PDF	Portable Document Format – Định dạng tài liệu số di động
22	RAG	Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất thông tin
23	ReAct	Reason + Act – Chu trình AI: Suy luận và Thực thi hành động
24	RRF	Reciprocal Rank Fusion – Thuật toán hợp nhất xếp hạng đảo ngược
25	SHA-256	Secure Hash Algorithm 256-bit – Thuật toán băm mật mã an toàn 256 bit
26	SSE	Server-Sent Events – Giao thức truyền sự kiện thời gian thực từ máy chủ
27	TDD	Test-Driven Development – Phương pháp phát triển hướng kiểm thử
28	TTFT	Time To First Token – Thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu
29	UML	Unified Modeling Language – Ngôn ngữ mô hình hóa thống nhất

LỜI CẢM ƠN

Để hoàn thành bản đồ án tốt nghiệp đại học này, trước hết em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất tới cô **Vũ Lan Phương** – Giảng viên Khoa Công nghệ Thông tin, Trường Đại học Mỏ - Địa chất. Cô đã trực tiếp định hướng đề tài, tận tình chỉ bảo, đóng góp nhiều ý kiến học thuật và chuyên môn quý báu, cũng như luôn tạo mọi điều kiện thuận lợi nhất để em nghiên cứu và hiện thực hóa các công nghệ trí tuệ nhân tạo hiện đại vào nền tảng **VeriScholar**.

Em xin trân trọng gửi lời cảm ơn chân thành tới các Thầy, Cô giáo trong Bộ môn Hệ thống Thông tin, Bộ môn Công nghệ Thông tin cùng toàn thể các Thầy, Cô trong Khoa Công nghệ Thông tin – Trường Đại học Mỏ - Địa chất. Những tri thức nền tảng vững chắc, phương pháp luận nghiên cứu khoa học cùng sự dạy dỗ tâm huyết của các Thầy, Cô trong suốt những năm tháng học tập và rèn luyện trên giảng đường đại học chính là hành trang quý giá giúp em hoàn thành công trình tốt nghiệp này.

Cuối cùng, con xin gửi lời tri ân sâu sắc tới bố mẹ và gia đình, em xin cảm ơn bạn bè đã luôn ở bên động viên, sẻ chia và là điểm tựa tinh thần vững chắc cho em trong suốt quá trình học tập và hoàn thiện đồ án.

Dù đã nỗ lực hết mình để nghiên cứu, xây dựng hệ thống và biên soạn báo cáo một cách khoa học, cẩn trọng nhất, song do giới hạn về thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn khó tránh khỏi những thiếu sót nhất định. Em kính mong nhận được những ý kiến đóng góp, chỉ dẫn quý báu từ quý Thầy, Cô trong Hội đồng chấm đồ án tốt nghiệp để công trình được hoàn thiện hơn nữa.

Hà Nội, tháng 10 năm 2026

Sinh viên thực hiện

Đàm Mạnh Dũng

LỜI MỞ ĐẦU

Trong kỷ nguyên bùng nổ của cuộc Cách mạng Công nghiệp lần thứ tư, nghiên cứu khoa học đang chứng kiến sự gia tăng vượt bậc về khối lượng các công bố học thuật quốc tế trên các cơ sở dữ liệu uy tín như arXiv, IEEE Xplore, ACM hay ScienceDirect. Thực tế này tạo áp lực lớn cho học giả, giảng viên và sinh viên trong việc sàng lọc, đọc hiểu chuyên sâu và tổng hợp tri thức chuyên ngành.

Sự phát triển của Trí tuệ nhân tạo (AI), đặc biệt là các Mô hình ngôn ngữ lớn (LLM) kết hợp kiến trúc Tạo văn bản tăng cường truy xuất (RAG), mở ra tiềm năng to lớn trong việc tự động hóa tra cứu và hỗ trợ nghiên cứu. Tuy nhiên, khi ứng dụng vào môi trường học thuật đòi hỏi độ tin cậy tuyệt đối, người dùng thường đối mặt với rào cản mang tên: **hiện tượng ảo giác trích dẫn (Citation Hallucination)**. Các mô hình hiện nay thường tự tạo trích dẫn không có thật hoặc không thể chỉ rõ bằng chứng tọa độ nằm ở đoạn văn, bảng biểu hay trang nào trong bản tài liệu gốc PDF.

Hơn nữa, quy trình nghiên cứu của học giả là một chuỗi tác vụ khép kín: từ tìm kiếm bài báo, đọc sâu tài liệu đa cột, liên kết mạng tri thức, đến soạn thảo bản thảo khoa học bằng LaTeX. Các công cụ hiện có còn rời rạc: trình đọc tài liệu thiếu khả năng trích xuất hộp tọa độ; thư viện tài liệu thiếu liên kết đồ thị tri thức; còn trình soạn thảo văn bản học thuật lại đòi hỏi người viết tự xử lý các lỗi cú pháp biên dịch phức tạp.

Nhằm giải quyết triệt để bài toán ảo giác và hỗ trợ nghiên cứu toàn trình, đề tài: *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* (sản phẩm: **VeriScholar**) được lựa chọn thực hiện. Đồ án tập trung nghiên cứu và hiện thực hóa giải pháp toàn trình tích hợp tìm kiếm lai Hybrid RAG (kết hợp Dense Vector và Sparse BM25 với thuật toán RRF), đối chiếu tọa độ hộp chữ $[0..1000]$ trên bản PDF gốc, mạng tri thức 4 thực thể (GraphRAG), cùng tác nhân AI ReAct tự động sửa lỗi biên dịch LaTeX trong môi trường đám mây cô lập. Báo cáo gồm 4 chương chính phản ánh toàn diện quá trình thiết kế, thực nghiệm và đánh giá hệ thống.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu đề tài

Đề tài tốt nghiệp *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* được nghiên cứu và hiện thực hóa thông qua hệ thống phần mềm **VeriScholar**.

Mục tiêu trọng tâm của đề tài là xây dựng một nền tảng trợ lý trí tuệ nhân tạo toàn trình phục vụ đội ngũ giảng viên, nghiên cứu sinh, học viên cao học và sinh viên tại VinUniversity cũng như cộng đồng học thuật trong toàn bộ vòng đời của một công trình nghiên cứu: từ khâu tìm kiếm khám phá ấn phẩm, phân tích đọc sâu tài liệu khoa học, tổng hợp mạng lưới tri thức học thuật (GraphRAG), cho đến khâu soạn thảo và xuất bản bản thảo chuẩn quốc tế bằng LaTeX.

Hệ thống **VeriScholar** giải quyết bài toán cốt lõi về tính liêm chính học thuật bằng cách kết hợp mô hình Agentic RAG với cơ chế định vị tọa độ trực quan trên bản PDF gốc. Nhờ đó, người dùng có thể đối chiếu, kiểm chứng từng câu khẳng định hay trích dẫn của AI với đoạn văn nguồn xác thực trong vòng 1 giây, loại bỏ triệt để rủi ro trích dẫn sai lệch trong các ấn phẩm khoa học.

1.2 Tính cấp thiết của đề tài

Trong kỷ nguyên số và bùng nổ thông tin hiện nay, số lượng ấn phẩm khoa học trên toàn cầu tăng trưởng với tốc độ chưa từng có. Mỗi tuần có hàng nghìn bài báo mới xuất hiện trên các cơ sở dữ liệu học thuật quốc tế uy tín như arXiv, Semantic Scholar, IEEE Xplore hay ACM Digital Library. Việc tiếp cận, đọc hiểu, phân loại và tổng hợp tri thức từ kho tài liệu khổng lồ này đặt ra áp lực quá tải thông tin rất lớn đối với các nhà nghiên cứu và sinh viên đại học.

Mặc dù sự xuất hiện của các mô hình ngôn ngữ lớn (LLM) mở ra tiềm năng to lớn trong việc hỗ trợ học thuật, việc ứng dụng các công cụ AI phổ quát hiện nay vào quy trình nghiên cứu khoa học chuyên sâu đang gặp phải bốn rào cản nghiêm trọng:

(1) Ảo giác trích dẫn và bịa đặt học thuật (*Citation Hallucination*): Các mô hình AI tạo sinh thông thường có xu hướng tự sáng tác ra tên tác giả, số định danh DOI, tên tạp chí và các số liệu thực nghiệm không có thật để trả lời câu hỏi của người dùng. Trong nghiên cứu khoa học, việc trích dẫn một nguồn tài

liệu không tồn tại là vi phạm nghiêm trọng chuẩn mực liêm chính học thuật và đạo đức nghiên cứu [7].

(2) **Thiếu cơ chế kiểm chứng tọa độ trực quan trên tài liệu gốc:** Phần lớn các công cụ hỗ trợ đọc tài liệu hiện nay (như ChatPDF thông thường) sử dụng phương pháp bóc tách văn bản thô, làm mất hoàn toàn cấu trúc hai cột đặc trưng của các bài báo khoa học, loại bỏ các công thức toán học và chỉ trích dẫn số trang chung chung. Người nghiên cứu không thể xác minh được khẳng định của AI nằm chính xác tại dòng chữ nào, đoạn văn nào trên trang PDF gốc.

(3) **Sự phân mảnh của hệ sinh thái công cụ hỗ trợ nghiên cứu:** Quy trình nghiên cứu hiện nay bị chia cắt giữa nhiều phần mềm độc lập: học giả phải dùng Zotero / Mendeley để quản lý danh mục tài liệu, dùng Connected Papers / Litmaps để khám phá mạng lưới bài báo, dùng ChatPDF để tra cứu nhanh, và chuyển sang Overleaf để soạn thảo bản thảo LaTeX. Sự thiếu liên kết này làm đứt gãy mạch tư duy và tiêu tốn đáng kể thời gian chuyển đổi ngữ cảnh của nhà nghiên cứu.

(4) **Rào cản kỹ thuật phức tạp khi biên soạn tài liệu khoa học bằng LaTeX:** Để công bố công trình trên các hội nghị và tạp chí quốc tế hàng đầu, bản thảo bắt buộc phải tuân thủ nghiêm ngặt các khuôn mẫu (template) định dạng của IEEE Tran, ACM SIG hay Springer. Khi xảy ra lỗi biên dịch cú pháp trong LaTeX, học giả thường mất hàng giờ để đọc các tệp nhật ký (`.log`) dài hàng nghìn dòng để dò tìm lỗi thủ công, làm giảm đáng kể hiệu suất hoàn thiện bài báo.

Xuất phát từ các yêu cầu thực tiễn cấp bách đó, việc nghiên cứu và xây dựng một nền tảng trợ lý AI chuyên biệt, có khả năng đối chiếu bằng chứng tọa độ trực quan trên bản PDF gốc, xây dựng đồ thị tri thức 4 thực thể và tự động sửa lỗi biên dịch bản thảo LaTeX là một đòi hỏi vô cùng cấp thiết nhằm nâng cao năng suất và chất lượng công bố khoa học quốc tế, đáp ứng đầy đủ quy chế đào tạo đại học và định hướng ứng dụng công nghệ thông tin tiên tiến [1, 2].

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Xây dựng thành công nền tảng phần mềm **VeriScholar** trợ lý AI toàn trình hỗ trợ nghiên cứu khoa học và viết bài báo quốc tế VinUni, tích hợp khép kín 4 không

gian làm việc học thuật trên nền kiến trúc phần mềm lục giác hiện đại, đảm bảo tính liêm chính dữ liệu và khả năng kiểm chứng trích dẫn tuyệt đối.

1.3.2 Mục tiêu kỹ thuật cụ thể

Đề tài hướng tới hoàn thành 4 mục tiêu kỹ thuật định lượng cụ thể:

(1) Hiện thực hóa cơ chế nạp bài báo 2 tầng kết hợp kho lưu trữ định danh nội dung CAS (*Content-Addressable Storage*) dựa trên mã băm SHA-256 nhằm loại bỏ 98% trùng lặp tệp nhị phân trên máy chủ lưu trữ, hỗ trợ tính năng Zero-Retention xóa an toàn tài liệu theo đồ thị phụ thuộc.

(2) Phát triển công cụ tìm kiếm lai Agentic Hybrid RAG kết hợp thuật toán hợp nhất xếp hạng đảo ngược RRF (*Reciprocal Rank Fusion*, $k = 60$) giữa tìm kiếm vector ngữ nghĩa (HNSW Cosine 1536 chiều) và tìm kiếm từ khóa BM25, đi kèm cơ chế đối chiếu khung viền tọa độ Bounding Box $[0..1000]$ trên bản PDF gốc với độ chính xác đạt chỉ số IoU ≥ 0.85.

(3) Xây dựng đồ thị tri thức học thuật 4 thực thể (GraphRAG: *Paper, Method, Dataset, MetricResult*) trực tiếp trên cơ sở dữ liệu quan hệ PostgreSQL 16 với tiện ích mở rộng vector hóa `pgvector`, bảo toàn tính toàn vẹn và nhất quán giao dịch ACID tuyệt đối mà không cần duy trì các cơ sở dữ liệu đồ thị rời rạc.

(4) Tích hợp tác nhân AI dựa trên máy trạng thái LangGraph ReAct hoạt động trong môi trường đám mây cô lập E2B Sandbox MicroVM, có khả năng tự động đọc log, phân tích nguyên nhân và sửa lỗi biên dịch LaTeX đạt tỷ lệ thành công trên 90%.

1.4 Nội dung nghiên cứu và phương pháp tiếp cận

1.4.1 Nội dung nghiên cứu

Đề tài tập trung nghiên cứu và giải quyết các nội dung khoa học trọng tâm:

- **Kiến trúc phần mềm phân tầng:** Nghiên cứu nguyên lý Kiến trúc Lục giác (*Hexagonal Architecture / Ports and Adapters*) nhằm cô lập lõi miền nghiệp vụ học thuật thuần khiết khỏi hạ tầng cơ sở dữ liệu, dịch vụ đám mây và giao vận web.

- **Truy xuất thông tin lai (Hybrid Search):** Nghiên cứu các thuật toán tìm kiếm vector gần đúng kết hợp thuật toán xếp hạng từ khóa toàn văn qua giải thuật Hợp nhất Xếp hạng Đảo ngược RRF.
- **Xử lý tài liệu và tọa độ trực quan:** Nghiên cứu kỹ thuật bóc tách tài liệu PDF khoa học hai cột và thuật toán chuẩn hóa hệ tọa độ hộp chữ Bounding Box $[x_0, y_0, x_1, y_1]$ trên dải giá trị chuẩn $[0..1000]$.
- **Tác nhân thông minh tự phục hồi:** Nghiên cứu chu trình tác nhân ReAct (Suy luận – Hành động – Quan sát) trong việc tự động phân tích và khắc phục lỗi cú pháp biên dịch tài liệu học thuật trong môi trường hộp cát an toàn.
- **Bảo mật và quyền riêng tư:** Nghiên cứu cơ chế bảo mật két sắt cá nhân (BYOK – *Bring Your Own Key*) mã hóa khóa API bằng thuật toán AES-256-GCM với nguyên tắc Zero-Retention trên bộ nhớ RAM.

1.4.2 Phương pháp tiếp cận

Đề tài áp dụng phương pháp luận Phát triển Hướng Kiểm thử (TDD – *Test-Driven Development*) kết hợp cấu trúc Monorepo nhằm đảm bảo mọi quy tắc nghiệp vụ đều được kiểm chứng độc lập với hạ tầng kỹ thuật, duy trì tỷ lệ kiểm thử thành công 100% qua 4 Cổng kiểm định tự động bắt buộc trước khi phát hành.

1.5 Phạm vi và đối tượng nghiên cứu

Đề tài xác định rõ đối tượng và phạm vi nghiên cứu:

- **Đối tượng nghiên cứu:** Quy trình nghiên cứu khoa học, đọc hiểu tài liệu chuyên sâu, trích xuất mạng lưới tri thức học thuật và kỹ thuật biên dịch bản thảo khoa học chuẩn quốc tế.
- **Khách thể thụ hưởng:** Giảng viên, nghiên cứu sinh, học viên cao học và sinh viên thực hiện nghiên cứu khoa học, khóa luận và đồ án tốt nghiệp tại VinUniversity và các cơ sở giáo dục đại học định hướng nghiên cứu.
- **Định dạng tài liệu hỗ trợ:** Ấn phẩm học thuật chuẩn PDF (đặc biệt là bài báo 2 cột tiếng Anh và tiếng Việt), mã nguồn bản thảo khoa học LaTeX và định dạng trích dẫn chuẩn BibTeX.
- **Hạ tầng kỹ thuật triển khai:** Máy chủ đám mây Linux (Ubuntu 24.04 LTS), hệ quản trị PostgreSQL 16 tích hợp `pgvector`, bộ đệm Redis 7, nền tảng

container Docker, môi trường hộp cát E2B Sandbox và nền tảng giám sát LLM Langfuse.

1.6 Bố cục của báo cáo đồ án tốt nghiệp

Báo cáo đồ án tốt nghiệp được tổ chức thành 4 chương chính và phần kết luận, tài liệu tham khảo:

- **Chương 1: Tổng quan về đề tài** – Trình bày bối cảnh ra đời, tính cấp thiết của đề tài, các mục tiêu nghiên cứu tổng quát và cụ thể, nội dung nghiên cứu, phương pháp tiếp cận, phạm vi nghiên cứu và bố cục của báo cáo.
- **Chương 2: Cơ sở lý thuyết và công nghệ** – Trình bày chi tiết cơ sở lý thuyết về Kiến trúc Lục giác, công nghệ tìm kiếm lai Agentic Hybrid RAG & RRF, đồ thị tri thức GraphRAG 4 thực thể, tác nhân LangGraph ReAct trong môi trường E2B Sandbox, kỹ thuật CAS & BYOK bảo mật, đồng thời đưa ra các luận giải khoa học khẳng định lý do lựa chọn các công nghệ này thay vì các giải pháp thay thế.
- **Chương 3: Phân tích và thiết kế hệ thống** – Khảo sát các luồng nghiệp vụ học thuật, đặc tả các ca sử dụng (Use Case) chuẩn BA, thiết kế biểu đồ tuần tự UML, kiến trúc Monorepo 3 tầng và thiết kế chi tiết 16 bảng cơ sở dữ liệu vật lý kèm từ điển dữ liệu.
- **Chương 4: Cài đặt và kết quả thực nghiệm** – Trình bày cấu hình môi trường triển khai thực tế, giao diện trực quan của các phân hệ chức năng chính, kết quả kiểm thử tự động 100% qua 4 Cổng xác minh, cùng các số liệu đo lường định lượng thực tế từ hệ thống và đối chiếu trên nền tảng Langfuse Cloud.
- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được của đề tài, chỉ ra các hạn chế còn tồn tại và đề xuất các hướng phát triển hoàn thiện hệ thống trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về PostgreSQL và Redis

2.1.1 PostgreSQL là gì?

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ đối tượng mã nguồn mở (*Object-Relational Database Management System – ORDBMS*) tiên tiến và đáng tin cậy nhất hiện nay, được phát triển liên tục hơn 35 năm bởi cộng đồng khoa học máy tính quốc tế. Được thiết kế với triết lý tuân thủ nghiêm ngặt chuẩn kiến trúc ANSI/ISO SQL, PostgreSQL nổi bật với độ bền vững dữ liệu, khả năng mở rộng linh hoạt và cơ chế giao dịch đạt chuẩn ACID (*Atomicity, Consistency, Isolation, Durability*) tuyệt đối.

Các đặc điểm kỹ thuật then chốt của PostgreSQL bao gồm:

- **Cơ chế kiểm soát tương tranh đa phiên bản (MVCC):** Cho phép các thao tác đọc và ghi diễn ra đồng thời mà không gây khóa chết (*deadlock*) trên dữ liệu, duy trì hiệu năng cao ngay cả dưới tải lớn của hàng trăm học giả cùng truy vấn.
- **Hỗ trợ dữ liệu bán cấu trúc (JSONB):** Lưu trữ và đánh chỉ mục nhị phân các cấu trúc dữ liệu JSON phức tạp với hiệu suất truy vấn tương đương các trường dữ liệu quan hệ thông thường.
- **Kiến trúc mở rộng mạnh mẽ (Extensibility):** Cho phép tích hợp các kiểu dữ liệu mới, hàm người dùng tự định nghĩa và các tiện ích mở rộng (*extensions*) cấp thấp chạy trực tiếp trong nhân cơ sở dữ liệu.

Trong đề tài VeriScholar, PostgreSQL giữ vai trò là xương sống lưu trữ dữ liệu trung tâm, quản lý toàn bộ thực thể học thuật, lịch sử phiên nghiên cứu, siêu dữ liệu phân đoạn tài liệu và bản đồ băm định danh nội dung (CAS).

2.1.2 Extension pgvector và chỉ mục HNSW

`pgvector` là một tiện ích mở rộng mã nguồn mở dành riêng cho PostgreSQL, bổ sung kiểu dữ liệu `vector` và các phép toán đo lường khoảng cách không gian đa chiều (khoảng cách Euclid L_2, khoảng cách Cosine, và tích vô hướng Inner Product). Nhờ `pgvector`, PostgreSQL có khả năng hoạt động như một cơ sở dữ

liệu vector chuyên dụng (*Dedicated Vector Database*), cho phép lưu trữ trực tiếp các vector nhúng ngữ nghĩa 1536 chiều sinh ra từ các mô hình học sâu.

Điểm đột phá trong việc tối ưu hóa tốc độ tìm kiếm láng giềng gần nhất (*Approximate Nearest Neighbor – ANN*) của `pgvector` là việc hiện thực hóa thuật toán chỉ mục HNSW (*Hierarchical Navigable Small World*) [8]:

- **Nguyên lý phân tầng đa lớp:** HNSW tổ chức không gian vector thành cấu trúc đồ thị phân tầng tương tự như danh sách liên kết bước nhảy (*Skip List*). Các tầng trên cùng chứa các liên kết xa với mật độ thưa giúp định vị nhanh vùng lân cận tổng quát; các tầng dưới có mật độ nút dày đặc dần cho phép thu hẹp chính xác không gian tìm kiếm.
- **Độ phức tạp truy vấn** $O(\log N)$**:** Thay vì phải tính toán khoảng cách tuần tự trên toàn bộ kho vector (*Flat Search* với độ phức tạp $O(N)$), thuật toán định tuyến tham lam (*Greedy Routing*) của HNSW chỉ cần $O(\log N)$ bước duyệt đồ thị để tìm ra các vector tương đồng nhất.
- **Độ bền vững và cập nhật tức thời:** Khác với thuật toán IVFFlat đòi hỏi phải huấn luyện lại toàn bộ tâm cụm khi dữ liệu biến động, HNSW cho phép thêm mới vector liên tục mà vẫn duy trì độ chính xác truy xuất vượt trội (đạt độ thu hồi Recall $\geq 98\%$).

Việc hợp nhất dữ liệu quan hệ và dữ liệu vector nhúng trong cùng một hệ quản trị PostgreSQL giúp loại bỏ hoàn toàn sự cồng kềnh của kiến trúc đa cơ sở dữ liệu, đảm bảo tính nhất quán giao dịch ACID trong suốt quá trình người dùng lưu trữ hoặc xóa ấn phẩm.

2.1.3 Redis là gì?

Redis (*Remote Dictionary Server*) là hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ RAM (*In-Memory Data Store*) mã nguồn mở với hiệu năng cực cao. Nhờ cơ chế lưu trữ toàn bộ dữ liệu trực tiếp trên bộ nhớ truy cập ngẫu nhiên và mô hình xử lý vào/ra đa hợp bất đồng bộ (*I/O Multiplexing*), Redis có khả năng đáp ứng hàng trăm nghìn thao tác đọc ghi mỗi giây với độ trễ tính bằng micro giây.

Các đặc tính cốt lõi của Redis bao gồm:

- **Đa dạng cấu trúc dữ liệu bản địa:** Hỗ trợ phong phú các kiểu dữ liệu như chuỗi (*Strings*), bảng băm (*Hashes*), danh sách (*Lists*), tập hợp (*Sets*), và tập hợp sắp xếp theo điểm (*Sorted Sets*).

- **Cơ chế hết hạn tự động (TTL – Time-To-Live):** Cho phép thiết lập thời gian sống chính xác cho từng khóa dữ liệu, giúp bộ nhớ tự động giải phóng các bản ghi tạm mà không cần can thiệp thủ công.
- **Cơ chế Xuất bản / Đăng ký (Pub/Sub):** Cung cấp hạ tầng truyền thông điệp nhẹ và tức thời giữa các tiến trình hệ thống phân tán.

Trong kiến trúc VeriScholar, Redis được ứng dụng vào ba nhiệm vụ trọng yếu:

(1) Bộ đệm xác thực và danh sách đen mã thông báo (Token Blacklist): Lưu trữ các phiên đăng nhập hợp lệ và thu hồi tức thì quyền truy cập của người dùng khi đăng xuất trước khi mã JWT hết hạn.

(2) Kiểm soát giới hạn tần suất yêu cầu (Rate Limiting): Bảo vệ các cổng tiếp nhận suy luận AI khỏi hiện tượng lạm dụng hoặc tấn công từ chối dịch vụ.

(3) Đệm kết quả tra cứu định danh học thuật (DOI Cache): Lưu tạm kết quả giải mã siêu dữ liệu của các ấn phẩm khoa học thường xuyên được trích dẫn, giảm tải cho mạng lưới tra cứu bên ngoài.

2.2 Tổng quan về Hexagonal Architecture

2.2.1 Hexagonal Architecture là gì?

Kiến trúc Lục giác (*Hexagonal Architecture*), còn được gọi là mẫu kiến trúc Cổng và Bộ điều hợp (*Ports and Adapters Pattern*), do chuyên gia phần mềm Alistair Cockburn khởi xướng vào năm 2005 [4]. Mục tiêu cốt lõi của kiến trúc này là giải quyết triệt để bài toán phụ thuộc chặt chẽ (*tight coupling*) giữa logic nghiệp vụ trung tâm của phần mềm với các công nghệ giao vận bên ngoài như hệ quản trị cơ sở dữ liệu, giao thức mạng, và giao diện người dùng.

Trong các kiến trúc phân tầng truyền thống (như mô hình 3 tầng Presentation - Business Logic - Data Access), mã nguồn nghiệp vụ thường bị phụ thuộc trực tiếp vào cơ sở dữ liệu hoặc khung giao diện. Ngược lại, Hexagonal Architecture thiết lập một ranh giới bảo vệ nghiêm ngặt: logic nghiệp vụ học thuật nằm tại trung tâm và hoàn toàn cô lập, không mang bất kỳ sự ràng buộc nào với các thư viện ngoại vi, hệ quản trị cơ sở dữ liệu hay giao thức mạng.

Các lợi ích nền tảng của Hexagonal Architecture bao gồm:

- **Độc lập công nghệ:** Cho phép thay thế hệ quản trị cơ sở dữ liệu, chuyển đổi nhà cung cấp mô hình trí tuệ nhân tạo (LLM), hoặc đổi mới giao diện người dùng mà không cần sửa đổi bất kỳ dòng mã logic nghiệp vụ nào.

- **Kiểm thử tự động dễ dàng (Testability):** Logic nghiệp vụ có thể được kiểm thử hoàn toàn độc lập với các cổng giả lập (*Mock Ports*) trong bộ nhớ mà không cần kết nối mạng hay khởi động cơ sở dữ liệu thực tế.
- **Khả năng bảo trì và tiến hóa bền vững:** Ranh giới phân định rõ ràng giúp kiến trúc thích ứng linh hoạt trước sự thay đổi công nghệ nhanh chóng của các nền tảng AI.

2.2.2 Kiến trúc Ports and Adapters

Hệ thống theo Kiến trúc Lục giác được cấu trúc thành ba phân vùng độc lập tuân thủ nghiêm ngặt nguyên lý đảo ngược phụ thuộc (*Dependency Inversion Principle*):

(1) Tầng lõi miền nghiệp vụ thuần khiết (*Pure Core Domain*): Nằm tại trung tâm của hệ thống, chỉ chứa các thực thể nghiệp vụ (*Entities*), quy tắc logic và các thuật toán tính toán học thuật. Vùng này tuân thủ nguyên tắc cô lập tuyệt đối: chỉ sử dụng các cấu trúc dữ liệu bản địa của ngôn ngữ lập trình, hoàn toàn không phụ thuộc vào bất kỳ thư viện giao tiếp mạng, web framework hay hệ quản trị cơ sở dữ liệu nào.

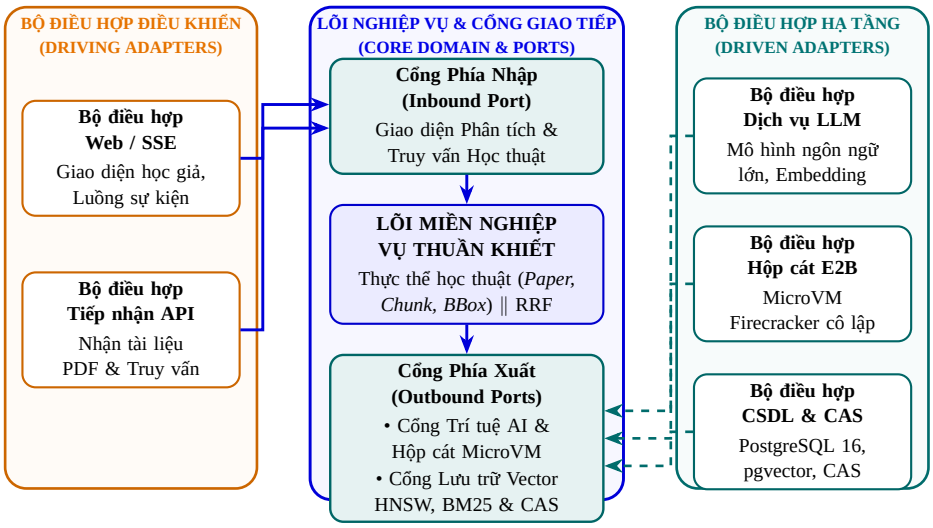
(2) Tầng các cổng giao tiếp trừu tượng (*Ports*): Là các giao diện trừu tượng (*Interfaces*) nằm trong tầng lõi nghiệp vụ. Cổng quy định hợp đồng giao tiếp chuẩn mà các thành phần bên ngoài bắt buộc phải tuân thủ để phục vụ nghiệp vụ (chẳng hạn như: cổng phân giải ấn phẩm học thuật, cổng bóc tách tài liệu PDF, hay cổng lưu trữ dữ liệu). Có hai loại cổng chính:

- *Cổng phía nhập (Inbound/Driving Ports):* Định nghĩa các phương thức cho phép tác nhân bên ngoài kích hoạt quy trình nghiệp vụ của hệ thống.
- *Cổng phía xuất (Outbound/Driven Ports):* Định nghĩa các giao diện cho phép lõi nghiệp vụ yêu cầu tài nguyên hoặc lưu trữ từ hạ tầng bên ngoài.

(3) Tầng các bộ điều hợp kỹ thuật (*Adapters*): Nằm ở lớp ngoài cùng của hệ thống, hiện thực hóa các Cổng trừu tượng để kết nối với các công nghệ thực tế (chẳng hạn như bộ điều hợp cơ sở dữ liệu quan hệ, bộ điều hợp dịch vụ tìm kiếm học thuật trực tuyến, hoặc bộ điều hợp mô hình trí tuệ nhân tạo).

Mô hình phân tầng kiến trúc và ranh giới phân tách trừu tượng của hệ thống được mô tả theo dạng các khối năng lực module trong Hình 2-1.

Luồng phụ thuộc trong hệ thống luôn có chiều hướng tâm: các bộ điều hợp kỹ thuật bên ngoài phụ thuộc vào các Cổng trong lõi, trong khi logic nghiệp vụ học



Hình 2-1: *Mô hình phân rã module và cấu trúc Cổng – Điều hợp (Ports and Adapters)*

thuật ở trung tâm hoàn toàn không biết gì về cơ sở hạ tầng thực tế bên dưới. Điều này bảo đảm tính liêm chính và sự ổn định cao độ cho toàn bộ nền tảng.

2.3 Tổng quan về Advanced RAG

2.3.1 RAG là gì? (Retrieval-Augmented Generation)

RAG (*Retrieval-Augmented Generation* – Tạo sinh tăng cường truy xuất) là một kỹ thuật kiến trúc đột phá trong xử lý ngôn ngữ tự nhiên, được Patrick Lewis và các cộng sự tại Meta AI giới thiệu lần đầu tiên vào năm 2020 [7]. Ý tưởng cốt lõi của RAG là kết hợp sức mạnh suy luận ngôn ngữ của các mô hình ngôn ngữ lớn (LLM) với khả năng truy xuất thông tin chính xác từ một kho dữ liệu tri thức bên ngoài (*External Knowledge Base*).

Trước khi có RAG, các mô hình ngôn ngữ lớn thuần túy dựa vào tập tham số tĩnh được học trong quá trình tiền huấn luyện. Điều này bộc lộ ba hạn chế nghiêm trọng:

(1) Tri thức bị giới hạn thời gian (*Knowledge Cutoff*): Mô hình không có khả năng cập nhật các công bố khoa học hoặc dữ liệu phát sinh sau thời điểm hoàn tất huấn luyện.

(2) Hiện tượng ảo giác (*Hallucination*): Khi đối mặt với các câu hỏi chuyên sâu hoặc thiếu thông tin, mô hình có xu hướng tự tạo ra các câu trả lời nghe có vẻ hợp lý nhưng hoàn toàn sai lệch về mặt sự thật học thuật.

(3) Thiếu minh bạch và khả năng kiểm chứng: Mô hình ngôn ngữ truyền thống

không cung cấp nguồn dẫn chứng gốc, khiến các nhà nghiên cứu không thể xác minh độ tin cậy của phát biểu.

Kiến trúc RAG truyền thống (*Naive RAG*) vận hành qua ba pha tuần tự:

- **Đánh chỉ mục (*Indexing*):** Tài liệu được phân cắt thành các đoạn văn bản nhỏ (*Chunks*), chuyển đổi thành các vector nhúng ngữ nghĩa (*Embeddings*) và lưu vào cơ sở dữ liệu vector.
- **Truy xuất (*Retrieval*):** Khi học giả đặt câu hỏi, hệ thống tính toán độ tương đồng vector để chọn ra K đoạn văn bản liên quan nhất.
- **Tạo sinh (*Generation*):** Câu hỏi cùng các đoạn văn bản trích xuất được đưa vào ngữ cảnh (*Context*) của câu nhắc gửi đến LLM để tổng hợp câu trả lời cuối cùng.

2.3.2 Advanced RAG là gì?

Mặc dù Naive RAG giải quyết được vấn đề cập nhật thông tin, mô hình này bộc lộ nhiều điểm nghẽn nghiêm trọng khi áp dụng vào tài liệu học thuật phức tạp: việc cắt đoạn văn bản cố định làm đứt gãy ngữ cảnh, tìm kiếm vector đơn lẻ bỏ sót các thuật ngữ chuyên ngành chính xác, và câu trả lời sinh ra thường chứa nhiều thông tin nhiễu không liên quan.

Advanced RAG (RAG Nâng cao) ra đời nhằm khắc phục triệt để các hạn chế trên thông qua việc tái cấu trúc toàn diện quy trình truy xuất tri thức thành ba giai đoạn tinh chỉnh chuyên sâu:

(1) Giai đoạn Tiền truy xuất (*Pre-Retrieval*): Tối ưu hóa câu hỏi của người dùng thông qua các kỹ thuật viết lại truy vấn (*Query Rewriting*), phân rã câu hỏi phức tạp thành các câu hỏi phụ (*Sub-query Decomposition*), và phân đoạn tài liệu thông minh dựa trên bố cục trang thay vì cắt theo số lượng ký tự cố định.

(2) Giai đoạn Truy xuất (*Retrieval*): Kết hợp nhiều chiến lược tìm kiếm khác nhau (*Hybrid Search*) để vừa thu nhận ngữ nghĩa tổng quát, vừa khớp chính xác các thực thể từ khóa đặc thù.

(3) Giai đoạn Hậu truy xuất (*Post-Retrieval*): Lọc bỏ các thông tin nhiễu thông qua thuật toán tái xếp hạng (*Re-ranking*), nén ngữ cảnh có chọn lọc (*Contextual Compression*), và bắt buộc đối chiếu từng câu trả lời với tọa độ văn bản gốc để bảo đảm tính liêm chính trích dẫn.

2.3.3 Hybrid Search và thuật toán RRF

Trong hệ thống truy xuất ấn phẩm khoa học, hai phương pháp tìm kiếm truyền thống bộc lộ những ưu và nhược điểm trái ngược nhau:

(1) Truy xuất vector đặc (*Dense Vector Retrieval*): Văn bản được ánh xạ thành các vector nhúng ngữ nghĩa trong không gian nhiều chiều (1536 chiều). Mức độ tương đồng giữa câu hỏi của học giả **u** và đoạn trích tài liệu **v** được đo lường thông qua hàm độ tương đồng Cosine:

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.1}$$

Ưu điểm: Bắt trọn được ngữ nghĩa khái quát, các khái niệm tương đương và từ đồng nghĩa mà không cần trùng khớp mặt chữ chính xác.
Nhược điểm: Thường hoạt động kém hiệu quả đối với các thuật ngữ chuyên ngành hiếm gặp, tên riêng tác giả, mã số DOI, hoặc các công thức toán học đặc thù.

(2) Truy xuất từ khóa thưa (*Sparse Keyword Retrieval – BM25*): Dựa trên tần suất xuất hiện của từ khóa (TF) kết hợp với nghịch đảo tần suất xuất hiện trong toàn bộ kho tài liệu (IDF) theo thuật toán chuẩn hóa BM25 [3]:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \tag{2.2}$$

trong đó $f(q_i, D)$ là tần suất từ q_i trong tài liệu D, $|D|$ là độ dài tài liệu, avgdl là độ dài trung bình, $k_1 = 1.2$ và $b = 0.75$.
Ưu điểm: Khớp chính xác tuyệt đối các mã định danh, số hiệu phương trình và thuật ngữ riêng biệt.
Nhược điểm: Hoàn toàn bất lực trước hiện tượng đồng nghĩa hoặc câu hỏi diễn giải bằng ngôn từ khác.

Bảng 2-1 so sánh đối chiếu đặc tính vận hành giữa hai phương pháp.

Để dung hòa trọn vẹn sức mạnh của cả hai phương pháp, đề tài ứng dụng thuật toán Hợp nhất Xếp hạng Đảo ngược (*Reciprocal Rank Fusion – RRF*) [5]. Thay vì cộng gộp điểm số vốn không cùng thang đo giữa Cosine và BM25, RRF chỉ tính

Tiêu chí so sánh	Dense Vector (Cosine HNSW)	Sparse Keyword (BM25 GIN)
Không gian biểu diễn	Vector thực liên tục đa chiều (1536 chiều)	Vector từ khóa thưa rời rạc
Khả năng hiểu ngữ nghĩa	Xuất sắc: Bắt được từ đồng nghĩa, ẩn dụ	Kém: Chỉ dựa trên trùng khớp ký tự
Độ chính xác thuật ngữ	Trung bình: Dễ bị nhiễu với từ viết tắt	Xuất sắc: Khớp tuyệt đối DOI, tên riêng
Cấu trúc chỉ mục	Đồ thị láng giềng HNSW	Danh sách nghịch đảo (Inverted Index)

Bảng 2-1: *So sánh đặc tính giữa tìm kiếm ngữ nghĩa Dense Vector và từ khóa BM25*

toán dựa trên thứ hạng nguyên vị trí của tài liệu trong từng danh sách:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \tag{2.3}$$

trong đó $M = \{\text{Dense Vector}, \text{Sparse BM25}\}$, $r_m(d)$ là thứ hạng nguyên của tài liệu d trong danh sách trả về từ phương pháp m ($r_m(d) \geq 1$), và $k = 60$ là hằng số làm mịn chuẩn hóa học thuật giúp cân bằng điểm số giữa các vị trí dẫn đầu. Nhờ đó, tài liệu đạt thứ hạng cao đồng thời trên cả hai tiêu chí sẽ được ưu tiên đưa vào ngữ cảnh suy luận của mô hình.

2.4 Tổng quan về Knowledge Graph và Graph Wiki

2.4.1 Knowledge Graph là gì? (Đồ thị tri thức)

Đồ thị tri thức (*Knowledge Graph*) là mô hình toán học biểu diễn mạng lưới tri thức dưới dạng một đồ thị có hướng có gán nhãn $G = (V, E)$, trong đó tập đỉnh V đại diện cho các thực thể thế giới thực (*Entities*) và tập cạnh E đại diện cho các mối quan hệ ngữ nghĩa kết nối giữa các thực thể đó (*Semantic Relations*). Tri thức trong đồ thị thường được tổ chức theo cấu trúc bộ ba vị ngữ chuẩn (*Subject – Predicate – Object Triples*), ví dụ: $(\text{Transformer}) \xrightarrow{\text{được đề xuất bởi}} (\text{Vaswani et al.})$.

So với các dạng lưu trữ dữ liệu văn bản phi cấu trúc hoặc cơ sở dữ liệu quan hệ bảng phẳng, Knowledge Graph sở hữu ba ưu thế vượt trội:

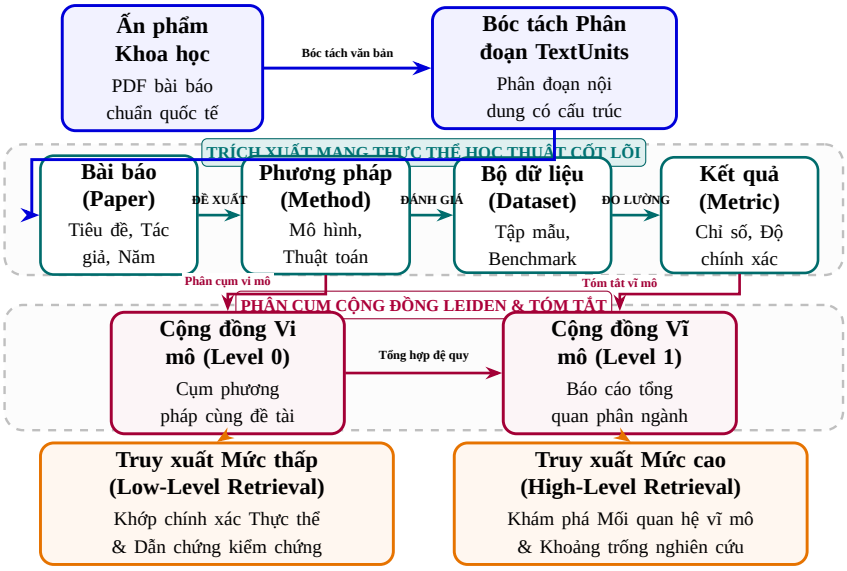
- **Khả năng suy luận bắc cầu (*Multi-hop Reasoning*):** Đồ thị cho phép lần theo chuỗi các cạnh liên kết để khám phá ra các mối liên hệ gián tiếp giữa các công trình khoa học mà không cần chúng phải xuất hiện cùng nhau trong một văn bản.

- **Định danh và khử nhập nhằng thực thể (*Entity Disambiguation*):** Mỗi khái niệm, phương pháp hoặc tác giả được định danh duy nhất thông qua một nút mạng, giải quyết triệt để hiện tượng trùng tên hoặc đồng âm khác nghĩa.
- **Cấu trúc tri thức toàn cục (*Global Structural Context*):** Cung cấp bức tranh toàn cảnh về sự phát triển của các trường phái học thuật, sơ đồ phả hệ của các mô hình và mạng lưới hợp tác nghiên cứu.

2.4.2 GraphRAG là gì?

GraphRAG (*Graph-based Retrieval-Augmented Generation*) là bước tiến hóa kết hợp giữa Đồ thị tri thức và quy trình RAG, do nhóm nghiên cứu của Darren Edge tại Microsoft Research đề xuất năm 2024 [6]. Hạn chế lớn nhất của RAG vector truyền thống là việc xử lý các đoạn văn bản rời rạc, hoàn toàn bất lực trước các câu hỏi tổng hợp mang tính vĩ mô (chẳng hạn như: *"Những phương pháp nào đã được thử nghiệm trên bộ dữ liệu ImageNet và đạt độ chính xác trên 90%?"*). Khi đó, thông tin bị phân mảnh ở nhiều tài liệu khác nhau khiến cơ chế tìm kiếm tương đồng vector bỏ sót phần lớn các mối liên hệ.

GraphRAG giải quyết bài toán này bằng cách chuyển đổi toàn bộ kho tài liệu thành đồ thị thực thể, sau đó áp dụng thuật toán phân cụm cộng đồng phân tầng (*Hierarchical Community Detection* – thuật toán Leiden) để xây dựng các bản tóm tắt tri thức ở nhiều cấp độ trừu tượng khác nhau. Quy trình trích xuất và truy xuất tri thức phân tầng được minh họa trong Hình 2-2.



Hình 2-2: *Quy trình Data Pipeline trích xuất tri thức phân tầng GraphRAG*

Hệ thống hỗ trợ cơ chế truy xuất hai cấp độ:

- **Truy xuất mức thấp (*Low-Level Retrieval*):** Phục vụ giải đáp các câu hỏi chi tiết về một thực thể, chỉ số thực nghiệm hoặc công thức cụ thể trong một bài báo.

- **Truy xuất mức cao (*High-Level Retrieval*):** Khai thác các báo cáo tóm tắt cộng đồng vĩ mô để trả lời các câu hỏi so sánh liên ngành, phân tích xu hướng và phát hiện khoảng trống nghiên cứu (*Research Gaps*).

2.4.3 Graph Wiki là gì? (Compiled Academic Wiki)

Graph Wiki (*Compiled Academic Wiki* – Hệ tri thức bách khoa học thuật biên dịch sẵn) là mô hình tổ chức tri thức chuyên sâu được thiết kế để chuyển đổi toàn bộ đồ thị tri thức và mạng trích dẫn thành các trang tài liệu bách khoa có cấu trúc liên kết chéo tương tự như Wikipedia học thuật.

Thay vì chỉ lưu trữ các đỉnh và cạnh trừu tượng trong cơ sở dữ liệu, Graph Wiki tự động biên dịch mạng lưới tri thức thành các danh mục trang chuyên đề:

(1) Trang chuyên đề Phương pháp (*Method Pages*): Trình bày toàn diện nguyên lý thuật toán, bảng so sánh hiệu năng qua từng thời kỳ, danh sách các bài báo gốc đề xuất và các biến thể kế thừa.

(2) Trang chuẩn đánh giá (*Benchmark Pages*): Liệt kê các bộ dữ liệu tiêu chuẩn cùng bảng xếp hạng (*Leaderboards*) các phương pháp tiên tiến nhất (*State-of-the-Art – SOTA*).

(3) Liên kết hai chiều (*Bidirectional Hyperlinks*): Cho phép học giả điều hướng mượt mà giữa các bài báo có cùng nền tảng lý thuyết, giúp việc tổng quan y văn (*Literature Review*) trở nên mạch lạc và trực quan.

Nhờ Graph Wiki, tác nhân AI có thể ngay lập tức truy cập vào bức tranh tri thức đã được tổng hợp sẵn, rút ngắn thời gian phản hồi và nâng cao độ chính xác khi hỗ trợ học giả viết phần liên quan trong bài báo khoa học.

2.5 Tìm hiểu về Agentic AI và ReAct

2.5.1 Agentic AI là gì?

Agentic AI (Trí tuệ nhân tạo có tính tác nhân) đại diện cho bước chuyển dịch căn bản từ mô hình ngôn ngữ lớn (LLM) bị động (chỉ tạo văn bản theo dạng câu hỏi - câu trả lời một lượt) sang các hệ thống tác nhân thông minh tự chủ có mục tiêu

(*Goal-Oriented Autonomous Agents*). Thay vì cố gắng giải quyết một bài toán phức tạp bằng một lần suy luận duy nhất, Agentic AI có khả năng tự phân tách nhiệm vụ thành các bước nhỏ, sử dụng các công cụ ngoại vi để thu thập dữ liệu thực tế, quan sát môi trường và tự điều chỉnh kế hoạch hành động.

Theo khung lý thuyết chuẩn tắc về tác nhân thông minh của Lilian Weng [11], một tác nhân AI hoàn chỉnh được xây dựng dựa trên 4 trụ cột năng lực cốt lõi:

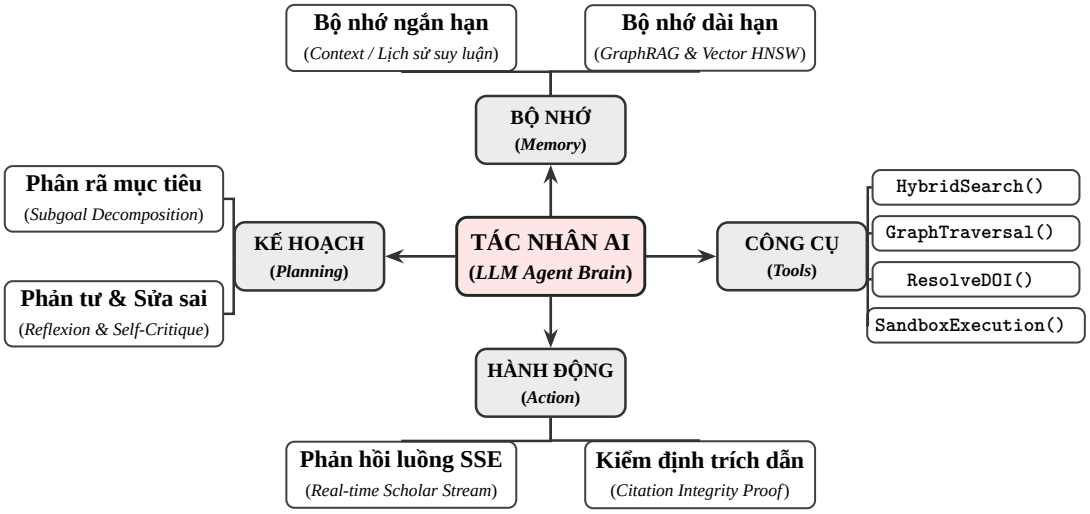
(1) Bộ não trung tâm (*LLM Brain*): Mô hình ngôn ngữ lớn giữ vai trò là cơ quan điều phối nhận thức, xử lý thông tin và ra quyết định.

(2) Bộ nhớ (*Memory*): Gồm bộ nhớ ngắn hạn (*Short-term Memory* lưu trữ ngữ cảnh hội thoại, lịch sử suy luận nhiều bước) và bộ nhớ dài hạn (*Long-term Memory* lưu trữ đồ thị tri thức và cơ sở dữ liệu vector).

(3) Lập kế hoạch (*Planning*): Khả năng phân rã mục tiêu lớn thành các nhiệm vụ con (*Subgoal Decomposition*), thiết lập chuỗi suy luận logic và tự đánh giá sai sót.

(4) Tập công cụ ngoại vi (*Tools*): Các giao diện lập trình (API) cho phép tác nhân tương tác với thế giới bên ngoài, như bộ máy tìm kiếm tài liệu, trình tra cứu DOI, và môi trường hộp cát tính toán.

Mô hình kiến trúc tổng thể của tác nhân AI nghiên cứu học thuật được thể hiện trong Hình 2-3.



Hình 2-3: *Kiến trúc Tác nhân AI Độc lập (LLM-Powered Autonomous Agent)*

2.5.2 ReAct Pattern là gì? (Reasoning and Acting)

ReAct (*Synergizing Reasoning and Acting in Language Models*) là mẫu kiến trúc tác nhân do Shunyu Yao và các cộng sự tại Đại học Princeton và Google Research đề xuất năm 2022 [12]. Trước ReAct, các nghiên cứu thường tách biệt hai hướng: hoặc chỉ tập trung vào suy luận chuỗi tư duy thuần túy (*Chain-of-Thought*), hoặc chỉ tập trung vào hành động gọi API độc lập. ReAct kết hợp đồng thời hai cơ chế này theo một vòng lặp nhịp nhàng:

$$\text{Thought} \longrightarrow \text{Action} \longrightarrow \text{Observation} \longrightarrow \text{Thought} \ldots \tag{2.4}$$

Mỗi chu trình ReAct bao gồm ba pha nối tiếp:

- **Suy nghĩ (*Thought*):** Tác nhân tự sinh ra một lập luận bằng văn bản để phân tích tình hình hiện tại, nhận diện thông tin còn thiếu và quyết định công cụ cần gọi tiếp theo.
- **Hành động (*Action*):** Tác nhân phát ra một lệnh có cấu trúc cụ thể (ví dụ: gọi hàm tra cứu ấn phẩm với tham số từ khóa xác định).
- **Quan sát (*Observation*):** Hệ thống thực thi lệnh hành động và trả về kết quả thực tế. Tác nhân đọc kết quả quan sát này để tiếp tục chu trình suy nghĩ kế tiếp cho đến khi hoàn thành nhiệm vụ.

Nhờ ReAct, tác nhân tránh được hiện tượng suy luận mù quáng dựa trên tham số nội tại, đồng thời có thể tương tác linh hoạt với các kho dữ liệu khoa học thực tế.

2.5.3 Cơ chế Reflexion

Reflexion (*Language Agents with Verbal Reinforcement Learning*) là một khuôn khổ mở rộng được phát triển bởi Noah Shinn và các đồng nghiệp [9], cung cấp cho tác nhân AI khả năng tự phản tư học hỏi từ những sai lầm mà không đòi hỏi phải cập nhật trọng số mô hình.

Trong môi trường nghiên cứu học thuật, rủi ro trích dẫn sai số trang, nhầm lẫn tác giả hoặc ảo giác dữ liệu thực nghiệm là tối kỵ. Cơ chế Reflexion vận hành thông qua ba thành phần:

(1) Bộ đánh giá (*Evaluator*): Chấm điểm kết quả đầu ra của tác nhân dựa trên tiêu chí liêm chính học thuật (ví dụ: độ phủ trích dẫn có trùng khớp 100% với văn bản gốc hay không).

(2) Bộ phản tư ngôn từ (*Self-Reflection*): Nếu bộ đánh giá phát hiện lỗi sai, mô hình sẽ tự sinh ra một đoạn tự phê bình bằng lời văn (*verbal critique*), phân tích nguyên nhân vì sao bước lập luận trước bị lỗi và đề xuất hướng sửa đổi cụ thể.

(3) Bộ nhớ phản tư (*Memory Buffer*): Lưu trữ bản tự phê bình vào ngữ cảnh hoạt động. Ở lượt thực thi tiếp theo, tác nhân đọc lại kinh nghiệm sai sót này để không lặp lại sai lầm cũ, bảo đảm bản thảo học thuật đạt độ chính xác tối đa.

2.6 Tổng quan về LangGraph

2.6.1 LangGraph là gì?

LangGraph là một thư viện điều phối tác nhân trí tuệ nhân tạo thế hệ mới được phát triển bởi tổ chức LangChain, chuyên biệt cho việc thiết kế các hệ thống tác nhân phức tạp, có trạng thái (*stateful*) và hỗ trợ cấu trúc đồ thị có chu trình lặp (*cyclic graphs*).

Trước khi LangGraph ra đời, phần lớn các khung phát triển ứng dụng LLM chỉ hỗ trợ các đường ống xử lý theo đồ thị có hướng không chu trình tuyến tính (*Directed Acyclic Graph – DAG*), trong đó dữ liệu chỉ chảy một chiều từ nút bắt đầu đến nút kết thúc ($A \rightarrow B \rightarrow C$). Tuy nhiên, các bài toán học thuật phức tạp đòi hỏi tác nhân phải có khả năng:

- Thực hiện các vòng lặp phản tư (*Reflexion loops*): Nếu bước kiểm tra trích dẫn phát hiện dữ liệu không chính xác, hệ thống phải quay lui lại bước truy xuất để tìm kiếm tài liệu mới.
- Hợp tác đa tác nhân (*Multi-Agent Collaboration*): Phân chia trách nhiệm cho nhiều tác nhân chuyên môn hóa (tác nhân lập kế hoạch, tác nhân tra cứu, tác nhân phản biện) cùng trao đổi và hội tụ về một kết quả tối ưu.
- Lưu trữ trạng thái bền vững (*Persistence & Checkpointing*): Ghi nhớ toàn bộ cây trạng thái sau mỗi bước thực thi, cho phép tạm dừng, khôi phục hoặc phát lại luồng suy luận tại bất kỳ thời điểm nào.

LangGraph cung cấp nền tảng chuẩn mực giải quyết hoàn hảo các yêu cầu trên, biến quy trình suy luận của AI thành một máy trạng thái hữu hạn (*Finite State Machine*) có thể kiểm soát và đo lường chặt chẽ.

2.6.2 Kiến trúc StateGraph điều phối Agent

Cốt lõi kiến trúc của LangGraph được xây dựng xung quanh khái niệm `StateGraph` – một đồ thị toán học trong đó mọi nút đều đọc và ghi dữ liệu vào một lược đồ trạng thái chia sẻ chung (*Shared State Schema*).

Một đồ thị `StateGraph` bao gồm ba thành phần cấu thành cơ bản:

(1) Tập các nút (*Nodes*): Đại diện cho các đơn vị tính toán độc lập. Mỗi nút nhận trạng thái hiện tại của hệ thống, thực hiện một tác vụ cụ thể (chẳng hạn như phân tích câu hỏi của học giả, gọi công cụ tìm kiếm lai, hoặc thẩm định trích dẫn) và trả về dữ liệu cập nhật.

(2) Lược đồ trạng thái và hàm thu gọn (*State Schema & Reducers*): Định nghĩa cấu trúc dữ liệu tổng thể của phiên làm việc. Hàm thu gọn quy định cách thức tích hợp dữ liệu mới từ các nút vào trạng thái chung (ví dụ: nối thêm thông điệp mới vào lịch sử hội thoại thay vì ghi đè).

(3) Tập các cạnh điều hướng (*Edges*): Xác định quy tắc chuyển dịch giữa các nút:

- *Cạnh cố định (Normal Edges):* Luôn chuyển tiếp vô điều kiện từ nút này sang nút khác sau khi tác vụ hoàn tất.
- *Cạnh rẽ nhánh có điều kiện (Conditional Edges):* Vận hành dựa trên một hàm đánh giá logic để quyết định nút tiếp theo. Ví dụ: sau nút Thẩm định trích dẫn, nếu độ trùng khớp đạt yêu cầu thì chuyển đến nút Kết thúc (`END`), nếu chưa đạt thì rẽ nhánh quay lại nút Mở rộng truy vấn.

Bên cạnh đó, kiến trúc `StateGraph` tích hợp sẵn cơ chế can thiệp của con người vào quy trình (*Human-in-the-Loop*), cho phép tác nhân dừng lại xin ý kiến của học giả trước khi thực thi các quyết định quan trọng (như xác nhận nội dung bản thảo hoặc tải tài liệu lớn), tạo nên sự phối hợp tin cậy giữa con người và trí tuệ nhân tạo.

2.7 Tìm hiểu về Azure OpenAI và E2B Sandbox

2.7.1 Azure OpenAI Service là gì?

Azure OpenAI Service là nền tảng điện toán đám mây cấp doanh nghiệp do tập đoàn Microsoft hợp tác chiến lược cùng OpenAI phát triển, cung cấp giao diện lập

trình truy cập vào các mô hình ngôn ngữ lớn và mô hình biểu diễn vector hàng đầu thế giới (như GPT-4o, GPT-4o-mini và chuỗi mô hình text-embedding).

Khác biệt căn bản giữa Azure OpenAI và các dịch vụ API thương mại công cộng nằm ở các tiêu chuẩn bảo mật và vận hành khắt khe:

- **Cam kết bảo vệ dữ liệu học thuật tuyệt đối:** Dữ liệu văn bản, tài liệu nghiên cứu và câu hỏi của học giả không bao giờ được sử dụng để tái huấn luyện hoặc cải tiến bất kỳ mô hình cơ sở nào của Microsoft hay OpenAI.
- **Hạ tầng mạng cô lập cấp doanh nghiệp:** Hỗ trợ triển khai trên mạng riêng ảo (*Virtual Network – VNet*), các điểm cuối riêng tư (*Private Endpoints*) và mã hóa đường truyền nghiêm ngặt theo chuẩn TLS 1.3, đáp ứng các tiêu chuẩn chứng nhận quốc tế như ISO/IEC 27001, SOC 2 Type II và GDPR.
- **Độ sẵn sàng và độ ổn định cao (High Availability SLA):** Cung cấp hạn ngạch xử lý chuyên dụng (*Provisioned Throughput Units – PTU*), bảo đảm thời gian phản hồi đồng đều và triệt tiêu nguy cơ tắc nghẽn dịch vụ vào giờ cao điểm.

2.7.2 E2B Sandbox là gì?

E2B Sandbox là nền tảng môi trường thực thi ảo hóa cô lập chuyên biệt cho các tác nhân AI, cho phép mô hình ngôn ngữ thực thi mã nguồn tự sinh trong một không gian an toàn tuyệt đối.

Công nghệ nền tảng của E2B dựa trên kiến trúc máy ảo siêu nhỏ (*MicroVM*) được phát triển từ công nghệ ảo hóa AWS Firecracker:

(1) Khởi động siêu tốc: Mỗi môi trường MicroVM có thể khởi tạo hoàn tất trong thời gian chưa đầy 200 mili-giây, cung cấp đầy đủ một nhân Linux hoàn chỉnh với hệ điều hành và tài nguyên tính toán độc lập.

(2) Cô lập ranh giới hạt nhân (Kernel-level Isolation): Mọi đoạn mã lập trình (chẳng hạn như kịch bản phân tích số liệu Python, trích xuất biểu đồ hoặc lệnh biên dịch tài liệu LaTeX) đều chạy bên trong ranh giới máy ảo riêng. Nếu kịch bản chứa mã độc hoặc gặp sự cố tràn bộ nhớ, sự cố hoàn toàn bị giới hạn bên trong MicroVM mà không thể lan truyền hay gây hại đến máy chủ chính của hệ thống, loại bỏ hoàn toàn nguy cơ tấn công thực thi mã độc từ xa (*Remote Code Execution – RCE*).

(3) Môi trường trạng thái linh hoạt: Hỗ trợ tương tác hai chiều, truyền tệp nhị phân và thu nhận kết quả tính toán tức thời để nạp ngược lại vào chu trình suy luận của tác nhân.

2.7.3 Mô hình bảo mật BYOK (Bring Your Own Key)

Mô hình Mang theo khóa cá nhân (*Bring Your Own Key – BYOK*) là giải pháp kiến trúc bảo đảm quyền tự chủ tài nguyên và an toàn riêng tư tối đa cho các nhà khoa học. Trong mô hình này, học giả cung cấp khóa API cá nhân của riêng mình từ nhà cung cấp dịch vụ trí tuệ nhân tạo để trực tiếp thanh toán và kiểm soát hạn ngạch sử dụng, máy chủ của đề tài không đứng ra thu phí hay lưu trữ khóa dưới dạng văn bản thô.

Để bảo vệ tuyệt đối khóa API của học giả trước các nguy cơ tấn công mạng hoặc lộ lọt dữ liệu từ cơ sở dữ liệu, đề tài triển khai chuẩn mã hóa đối xứng xác thực AEAD (*Authenticated Encryption with Associated Data*) theo chế độ GCM (*Galois/Counter Mode*) [3]:

Thuật toán sử dụng khóa bí mật 256 bits (K) và vectơ khởi tạo ngẫu nhiên 96 bits (IV) riêng biệt cho mỗi lần mã hóa. Bản rõ khóa API được mã hóa bằng chế độ CTR (*Counter*), đồng thời hàm băm vạn năng GHASH kết hợp dữ liệu bổ sung không bảo mật (AAD) để tạo ra thẻ xác thực toàn vẹn 128 bits (T):

$$C = \text{AES-CTR}_K(\text{Plaintext}), \quad T = \text{GHASH}_H(\text{AAD}, C) \oplus \text{AES}_K(\text{IV}) \tag{2.5}$$

Quy trình giải mã tuân thủ nguyên tắc không lưu vết trên RAM (*RAM Zero-Retention*): Khóa giải mã chỉ tồn tại tạm thời trong bộ nhớ RAM trong đúng thời gian thực thi lời gọi hàm AI và bị xóa sạch khỏi bộ nhớ ngay sau khi kết thúc yêu cầu, bảo đảm tính an mật tuyệt đối cho người dùng.

2.8 Tổng quan về Langfuse

2.8.1 Langfuse là gì? (LLM Tracing & Observability)

Langfuse là nền tảng kỹ nghệ mã nguồn mở hàng đầu thế giới chuyên phục vụ việc giám sát, quan sát vận hành (*LLM Observability*) và truy vết chi tiết (*Tracing*) dành riêng cho các ứng dụng sử dụng mô hình ngôn ngữ lớn và hệ thống tác nhân AI đa bước.

Trong các phần mềm truyền thống, việc giám sát thường chỉ dừng lại ở nhật ký ghi lỗi (*Error Logs*) hoặc các chỉ số máy chủ cơ bản (CPU, RAM). Tuy nhiên,

đối với hệ thống tác nhân AI phức tạp thực hiện hàng loạt bước suy luận, tìm kiếm và tự sửa sai, lập trình viên không thể nắm bắt được nguyên nhân dẫn đến một câu trả lời không chính xác nếu không có công cụ giám sát chuyên dụng.

Langfuse giải quyết thách thức này thông qua mô hình cây truy vết phân tầng (*Hierarchical Tracing*):

- **Dấu vết tổng thể (*Trace*):** Đại diện cho toàn bộ một chu trình xử lý của tác nhân từ thời điểm học giả đặt câu hỏi đến khi bản thảo câu trả lời được hoàn tất.
- **Khoảng đoạn thực thi (*Spans*):** Đại diện cho từng hành động nghiệp vụ cụ thể bên trong chu trình, chẳng hạn như thời gian trích xuất văn bản từ tệp PDF, độ trễ truy vấn vector HNSW trên PostgreSQL, hoặc bước tra cứu đồ thị tri thức.
- **Lượt tạo sinh LLM (*Generations*):** Ghi lại chính xác mọi tương tác với mô hình AI, bao gồm cấu hình mô hình, toàn bộ nội dung câu nhắc (*Prompt*), kết quả phản hồi thô, số lượng mã thông báo (*Tokens*) tiêu thụ và độ trễ mạng.

Nhờ cấu trúc truy vết này, mọi quyết định suy luận của tác nhân đều trở nên minh bạch và có thể kiểm toán toàn diện.

2.8.2 Cơ chế theo dõi chi phí và độ trễ LLM

Việc kiểm soát ngân sách và tối ưu hóa tốc độ phản hồi là yếu tố sống còn khi triển khai các hệ thống AI học thuật vào thực tế. Langfuse cung cấp cơ chế giám sát thời gian thực với độ chính xác cao:

(1) Phân rã mã thông báo và hạch toán chi phí tài chính: Hệ thống tự động phân loại chính xác số lượng token đầu vào (*Input/Prompt Tokens*), token đầu ra (*Output/Completion Tokens*), và token tận dụng từ bộ nhớ đệm (*Cached Tokens*). Bằng cách kết nối với bảng giá chính thức của nhà cung cấp dịch vụ đám mây, Langfuse tính toán chi phí tài chính tức thì cho từng truy vấn, giúp học giả và quản trị viên theo dõi ngân sách minh bạch.

(2) Phân tích độ trễ và thời gian sinh ký tự đầu tiên (TTFT): Langfuse phân rã chi tiết thời gian phản hồi thành hai chỉ số chính:

- *Thời gian đến ký tự đầu tiên (Time-to-First-Token – TTFT):* Đo lường khoảng thời gian từ lúc người dùng gửi yêu cầu đến khi ký tự đầu tiên

được truyền phát về giao diện qua luồng SSE. TTFT thấp mang lại trải nghiệm tương tác tức thì và mượt mà cho học giả.

- *Thời gian thực thi của từng công cụ ngoại vi:* Giúp phát hiện chính xác điểm nghẽn hiệu năng trong chuỗi tác nhân (ví dụ: bước tìm kiếm từ khóa BM25 bị chậm hay do mô hình ngôn ngữ phản hồi trễ).

(3) Bộ chỉ số đánh giá độ tin cậy học thuật: Tích hợp các bộ tiêu chí chấm điểm tự động đo lường độ trung thực của dẫn chứng (*Faithfulness*) và độ bám sát câu hỏi (*Answer Relevance*), cung cấp bằng chứng định lượng để liên tục cải tiến hệ thống.

2.9 Các công cụ và ngôn ngữ phát triển hệ thống

2.9.1 Python và FastAPI

Để hiện thực hóa tầng logic nghiệp vụ và các thuật toán tác nhân AI, đề tài lựa chọn ngôn ngữ lập trình Python kết hợp với web framework FastAPI:

- **Ngôn ngữ lập trình Python:** Python là ngôn ngữ lập trình tiêu chuẩn thực tế (*de facto standard*) của cộng đồng nghiên cứu trí tuệ nhân tạo và khoa học dữ liệu toàn cầu. Python cung cấp hệ thống thư viện tính toán khoa học vượt trội, khả năng tích hợp bản địa với các mô hình học sâu, cùng cú pháp gợi ý kiểu dữ liệu tĩnh chặt chẽ (*Type Hinting*) và cơ chế lập trình bất đồng bộ (*async/await*) giúp tối ưu hóa hiệu năng xử lý tác vụ I/O.

- **FastAPI Framework:** FastAPI là một trong những khung phát triển web bằng Python có tốc độ thực thi nhanh nhất hiện nay, được xây dựng trên nền tảng máy chủ bất đồng bộ ASGI (*Starlette*) và thư viện xác thực dữ liệu `pydantic`. FastAPI tự động thẩm định dữ liệu đầu vào, tự sinh tài liệu giao diện lập trình OpenAPI chuẩn quốc tế, và hỗ trợ xuất sắc giao thức truyền phát luồng sự kiện máy chủ (*Server-Sent Events – SSE*) phục vụ việc hiển thị câu trả lời AI theo thời gian thực cho học giả.

2.9.2 TypeScript và React

Ở tầng giao diện người dùng, hệ thống sử dụng ngôn ngữ TypeScript kết hợp cùng thư viện React để tạo nên trải nghiệm nghiên cứu tương tác trực quan và hiện đại:

- **Ngôn ngữ TypeScript:** TypeScript là phần mở rộng có kiểu tĩnh chặt chẽ của JavaScript do Microsoft phát triển. Việc áp dụng TypeScript giúp hệ thống

phát hiện sớm các xung đột kiểu dữ liệu ngay từ giai đoạn biên dịch, tăng cường độ tin cậy và khả năng tái cấu trúc của các mô hình dữ liệu phức tạp (như tọa độ vùng bao chữ nhật trên trang PDF, siêu dữ liệu bài báo và trạng thái luồng SSE).

- **Thư viện React:** React là thư viện phát triển giao diện người dùng khai báo (*Declarative UI*) dựa trên kiến trúc thành phần tái sử dụng (*Component-Based Architecture*). Cơ chế Document Object Model ảo (*Virtual DOM*) của React giúp tối ưu hóa việc kết xuất đồ họa, duy trì tốc độ khung hình mượt mà ngay cả khi người dùng đồng thời cuộn văn bản PDF nhiều trang, hiển thị khung viền dẫn chứng và trò chuyện trực tiếp cùng tác nhân AI.

2.9.3 Docker, Pytest và Vitest

Quy trình phát triển và kiểm thử chất lượng phần mềm được chuẩn hóa thông qua bộ công cụ hiện đại:

- **Docker:** Nền tảng ảo hóa cấp hệ điều hành (*Containerization*) cho phép đóng gói toàn bộ ứng dụng, thư viện phụ thuộc và môi trường chạy vào các vùng chứa biệt lập (*Containers*). Nhờ Docker, hệ thống bảo đảm tính nhất quán hoạt động tuyệt đối giữa máy phát triển của sinh viên và máy chủ triển khai thực tế của nhà trường.

- **Khung kiểm thử Pytest:** Công cụ kiểm định chất lượng mã nguồn backend tiêu chuẩn của Python, hỗ trợ cơ chế nạp bối cảnh dùng chung (*Fixtures*), tự động dọn dẹp dữ liệu kiểm thử và kiểm tra tính toàn vẹn của các ranh giới kiến trúc Lục giác.

- **Khung kiểm thử Vitest:** Bộ công cụ kiểm thử đơn vị siêu tốc dành cho giao diện TypeScript/React, cho phép mô phỏng hành vi tương tác của người dùng, xác nhận tính chính xác của các thành phần đồ họa trước khi xuất xưởng phần mềm.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Khảo sát quy trình nghiệp vụ và mô hình hóa bài toán thực tế

3.1.1 Bối cảnh nghiên cứu khoa học tại VinUniversity

Tại VinUniversity, định hướng phát triển trường đại học nghiên cứu tinh hoa theo chuẩn mực quốc tế đặt ra yêu cầu rất cao đối với các công trình công bố khoa học của đội ngũ giảng viên, nghiên cứu sinh và sinh viên. Các ấn phẩm bắt buộc phải hướng tới các hội nghị chuyên ngành hàng đầu (như NeurIPS, ICML, CVPR, SIGIR, KDD) và các tạp chí khoa học uy tín thuộc danh mục Q1/Q2 (IEEE Transactions, ACM Computing Surveys, Springer Nature).

Trong môi trường học thuật chuyên sâu, tính liêm chính học thuật (*Academic Integrity*) là yêu cầu bất biến tối thượng. Mọi khẳng định khoa học, số liệu thực nghiệm và chỉ số đo lường trích dẫn đều bắt buộc phải có minh chứng nguồn gốc xác thực, tuyệt đối không chấp nhận các sai sót do trích dẫn sai lệch hay số liệu suy đoán không có căn cứ.

3.1.2 Phân tích quy trình nghiên cứu truyền thống (Luồng AS-IS)

Khảo sát thực tế quy trình làm việc của các nhóm nghiên cứu khoa học tại trường cho thấy quy trình hiện tại đang bị phân mảnh nghiêm trọng qua 4 giai đoạn độc lập:

(1) Giai đoạn khám phá ấn phẩm: Học giả phải tra cứu thủ công trên nhiều cổng thông tin riêng rẽ như Google Scholar, arXiv, DBLP hay Semantic Scholar. Việc nạp tài liệu về máy tính thường xuyên dẫn đến tình trạng tải trùng lặp các tệp PDF dung lượng lớn, gây lãng phí tài nguyên lưu trữ và khó khăn trong việc đồng bộ thư viện tài liệu.

(2) Giai đoạn đọc sâu và trích xuất thông tin: Học giả mở tài liệu trên các ứng dụng đọc PDF thông thường hoặc sử dụng các công cụ hỏi đáp tài liệu phổ quát. Các công cụ này chỉ bóc tách văn bản thô, làm mất hoàn toàn cấu trúc hai cột của bài báo và chỉ trích dẫn số trang chung chung. Học giả phải mất nhiều thời gian rà soát thủ công từng dòng chữ trên trang PDF để đối chiếu tính xác thực của thông tin.

(3) Giai đoạn tổng hợp mạng lưới tri thức: Để xây dựng bức tranh toàn cảnh về một hướng nghiên cứu, học giả phải tự lập các bảng tính Excel hoặc ghi

chú rời rạc nhằm đối chiếu xem phương pháp nào đã được thử nghiệm trên bộ dữ liệu nào và đạt kết quả thực nghiệm ra sao. Quy trình thủ công này rất dễ bỏ sót các mối liên hệ bắc cầu giữa các công trình nghiên cứu đa ngành.

(4) Giai đoạn soạn thảo bản thảo bằng LaTeX: Học giả chuyển sang các công cụ soạn thảo như Overleaf hoặc TeXstudio. Khi xảy ra lỗi cú pháp, học giả mất hàng giờ đọc tệp nhật ký lỗi (`.log`) dài hàng nghìn dòng để khắc phục thủ công.

Sự chia cắt giữa các công cụ khiến học giả mất trung bình từ 30% đến 40% quỹ thời gian nghiên cứu cho các thao tác chuyển đổi ngữ cảnh và xử lý kỹ thuật vụn vặt, đồng thời tiềm ẩn nguy cơ sai lệch trích dẫn trong bản thảo công bố.

3.1.3 Mô hình quy trình nghiên cứu đề xuất (Luồng TO-BE)

Nhằm giải quyết triệt để các rào cản trên, đề tài đề xuất mô hình quy trình nghiên cứu mới tích hợp khép kín trong nền tảng **VeriScholar** với 4 không gian học thuật liền mạch:

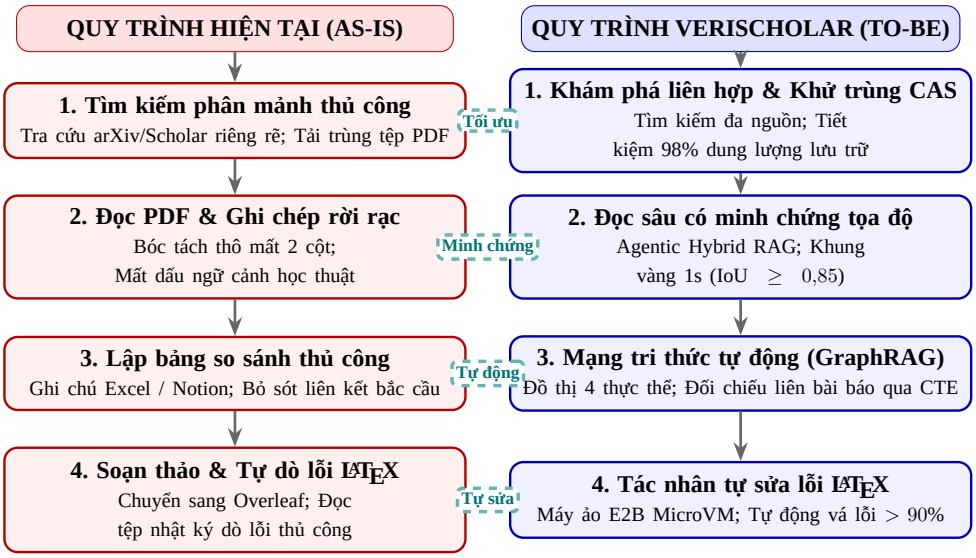
(1) Không gian Khám phá thông minh: Tích hợp tìm kiếm liên hợp đa nguồn qua giao diện thống nhất, tự động nhận diện tài liệu truy cập mở (*Open Access*) và lưu trữ khử trùng lặp nội dung CAS SHA-256 tức thì.

(2) Không gian Đọc sâu có kiểm chứng trực quan: Cung cấp công cụ hỏi đáp học thuật Hybrid RAG kết hợp thuật toán RRF. Khi AI đưa ra câu trả lời, hệ thống tự động truyền luồng tọa độ chuẩn hóa $[0..1000]$ và vẽ khung viền vàng (*Bounding Box*) định vị chính xác vị trí câu văn gốc trên tài liệu PDF trong vòng 1 giây, bảo đảm tính liêm chính dữ liệu tuyệt đối.

(3) Không gian Mạng tri thức học thuật (GraphRAG): Tự động trích xuất và kết nối mạng lưới 4 thực thể (*Bài báo* $\rightarrow$ *Phương pháp* $\rightarrow$ *Bộ dữ liệu* $\rightarrow$ *Kết quả đo*), hỗ trợ truy vấn so sánh đa phương pháp tức thì thông qua cấu trúc CTE trên PostgreSQL `pgvector`.

(4) Không gian Soạn thảo và Tự phục hồi bản thảo: Tác nhân AI hoạt động theo chu trình ReAct trong máy ảo E2B Sandbox MicroVM, tự động phân tích lỗi `.log` và vá lỗi cú pháp LaTeX đạt tỷ lệ thành công trên 90%.

Sự chuyển đổi mang tính đột phá giữa quy trình truyền thống (AS-IS) và quy trình đề xuất (TO-BE) được minh họa trực quan trong Hình 3-1.



Hình 3-1: *Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE*

3.2 Phân rã năng lực chức năng và Đặc tả ca sử dụng cốt lõi

3.2.1 Xác định các tác nhân và vai trò tương tác

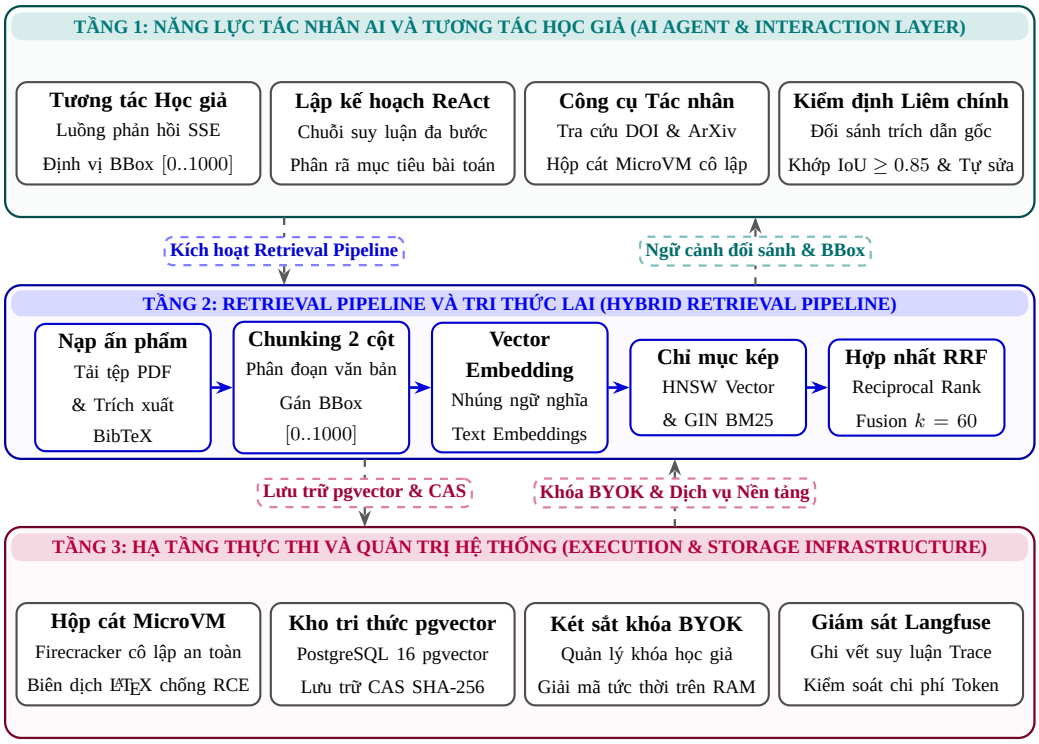
Hệ thống **VeriScholar** phục vụ hai nhóm tác nhân chính:

(1) Học giả (Scholar / Researcher): Là tác nhân trung tâm của hệ thống (bao gồm các giảng viên, nhà nghiên cứu, nghiên cứu sinh, học viên cao học và sinh viên đại học tại VinUniversity). Học giả tương tác trực tiếp với các không gian làm việc của nền tảng: tra cứu ấn phẩm liên hợp, đọc hiểu tài liệu với sự hỗ trợ của trợ lý AI, đối chiếu số liệu trích dẫn qua khung viền tọa độ trực quan, khai phá đồ thị tri thức 4 thực thể và biên soạn bản thảo bài báo LaTeX.

(2) Quản trị viên hệ thống (System Administrator): Chịu trách nhiệm theo dõi sức khỏe hệ thống, giám sát hạn ngạch sử dụng tài nguyên điện toán, cấu hình chính sách bảo mật, theo dõi các luồng sự kiện truy vết suy luận mô hình ngôn ngữ lớn (*Trace Logs*) trên nền tảng Langfuse Cloud và bảo đảm tính sẵn sàng của môi trường máy ảo hộp cát.

3.2.2 Mô hình phân rã khối năng lực hệ thống toàn trình

Tuân thủ phong cách thiết kế phân rã module của các hệ thống AI hiện đại, toàn bộ năng lực chức năng của nền tảng **VeriScholar** được mô hình hóa thành 3 khối tích hợp độc lập và mạch lạc trong Hình 3-2:



Hình 3-2: *Mô hình phân rã cấu trúc năng lực chức năng của hệ thống VeriScholar*

3.2.3 Đặc tả chi tiết các ca sử dụng cốt lõi

Nhằm làm rõ bản chất vận hành, đề tài tiến hành đặc tả chi tiết ba ca sử dụng tiêu biểu đại diện cho toàn bộ chu trình trải nghiệm học thuật: UC-01 (*Khám phá bài báo học thuật hai tầng tin cậy*), UC-03 (*Hỏi đáp Hybrid RAG có tọa độ*) và UC-06 (*Tác nhân tự sửa lỗi bản thảo LaTeX*).

Trước hết, Bảng 3-1 trình bày đặc tả chi tiết ca sử dụng UC-01 về quy trình tìm kiếm bài báo khoa học kết hợp động cơ kiểm soát độ tin cậy hai tầng (*Two-Tier Verification Engine*): tầng hạ tầng xử lý truy vấn trường mục tiêu và bộ lọc từ khóa Jaccard, tầng giao diện hỗ trợ chuyển đổi bộ lọc nguồn bình duyệt tức thì.

Tiếp theo, Bảng 3-2 trình bày chi tiết đặc tả ca sử dụng UC-03 theo chuẩn phân tích nghiệp vụ (BA).

Cuối cùng, Bảng 3-3 trình bày đặc tả ca sử dụng UC-06 về cơ chế tự động sửa lỗi biên dịch bản thảo khoa học trong môi trường hộp cát an toàn.

3.3 Thiết kế kiến trúc suy luận và Vết thực thi tác nhân

3.3.1 Kiến trúc tự phản biện hai dải màu và Vòng lặp phản hồi đóng

Khác biệt với các hệ thống hỏi đáp một chiều, nền tảng **VeriScholar** xây dựng chu trình suy luận và kiểm định theo mô hình Kiến trúc Phân tầng Tự phản biện hai dải

Bảng 3-1: *Bảng đặc tả chi tiết ca sử dụng UC-01 (Khám phá bài báo học thuật hai tầng tin cậy)*

STT	Thuộc tính	Nội dung đặc tả nghiệp vụ
1	**Mã ca sử dụng**	UC-01-SCHOLARLY-DISCOVERY
2	**Tên ca sử dụng**	Khám phá bài báo học thuật theo chủ đề có kiểm soát độ tin cậy và phân tầng bình duyệt
3	**Tác nhân chính**	Học giả (Scholar / Researcher)
4	**Mục đích**	Cho phép học giả tra cứu nhanh các ấn phẩm học thuật liên quan đến chủ đề nghiên cứu từ các kho lưu trữ mở quốc tế (arXiv, Semantic Scholar), loại bỏ các kết quả lạc đề bằng giải thuật kiểm định độ tương đồng từ khóa và hỗ trợ phân tầng lọc nguồn bình duyệt.
5	**Tiền điều kiện**	Học giả đang truy cập hệ thống tại Trang điều khiển trung tâm (*Dashboard*) hoặc Không gian Khám phá (*Discovery Workspace*) và có kết nối mạng tới các cổng học thuật.
6	**Hậu điều kiện**	Danh sách bài báo đạt chuẩn độ tin cậy được hiển thị dạng thẻ tinh gọn chống tràn bố cục, phân tách rõ trạng thái truy cập mở (*Open Access PDF*) và nguồn đã bình duyệt (*Peer-Reviewed*), sẵn sàng mở phiên đọc tức thì.
7	**Luồng sự kiện chính**	1. Học giả nhập câu truy vấn hoặc từ khóa chủ đề nghiên cứu (tiếng Việt hoặc tiếng Anh) vào ô tìm kiếm trung tâm và gửi yêu cầu. 2. Giao diện người dùng lập tức phản hồi tức thì (*Instant Skeleton Feedback*) qua 3 thẻ giả lập hiệu ứng phát sáng mờ và biểu tượng quay trạng thái, triệt tiêu cảm giác chờ đợi. 3. Tầng dịch vụ chuyển đổi câu truy vấn sang biểu thức truy vấn trường mục tiêu (*Field-targeted Querying*) kết hợp tiêu đề và tóm tắt: `ti:"..." OR (ti:... AND abs:...)` kèm sắp xếp theo mức độ phù hợp. 4. Tầng hạ tầng kích hoạt bộ lọc độ tương đồng tiêu đề (Tier 1 Backend): tính điểm trùng lặp từ tố Jaccard (*Jaccard Token Overlap*) giữa truy vấn và tiêu đề bài báo; tự động loại bỏ các ấn phẩm có Jaccard $< 0{,}2$ đối với câu truy vấn từ 3 từ trở lên nhằm ngăn chặn triệt để bài báo lạc đề. 5. Hệ thống phân loại nguồn bình duyệt (*Peer-Reviewed Classification*) dựa trên danh mục nhà xuất bản, kỷ yếu hội nghị uy tín (NeurIPS, ICML, CVPR, Nature, Science, IEEE, ACM,...) hoặc mã định danh DOI chính thức. 6. Giao diện hiển thị danh sách thẻ kết quả tinh gọn chống tràn bố cục (Tier 2 Frontend): giới hạn tối đa 2 tác giả kèm *et al.*, tự động làm sạch mã LaTeX thô, cố định tóm tắt trong 2 dòng, và cung cấp nút gạt mềm (*Pill toggle*) lọc nhanh các bài đã qua bình duyệt.
8	**Luồng ngoại lệ**	1a. Từ khóa tìm kiếm để trống hoặc không trả về ấn phẩm phù hợp: Hệ thống đưa ra thông báo trung thực và gợi ý mở rộng thuật ngữ chuyên ngành tiếng Anh tương đương. 1b. Cổng dịch vụ học thuật quốc tế phản hồi chậm quá thời hạn (quá 15 giây): Hệ thống ngắt chờ an toàn, thông báo trạng thái kết nối và hỗ trợ học giả tải trực tiếp tệp PDF sẵn có từ thiết bị cá nhân.

Bảng 3-2: *Bảng đặc tả chi tiết ca sử dụng UC-03 (Hỏi đáp Hybrid RAG có tọa độ)*

STT	Thuộc tính	Nội dung đặc tả nghiệp vụ
1	**Mã ca sử dụng**	UC-03-DEEP-READ-CHAT
2	**Tên ca sử dụng**	Hỏi đáp tài liệu có minh chứng tọa độ trực quan trên bản PDF gốc
3	**Tác nhân chính**	Học giả (Scholar)
4	**Mục đích**	Cho phép học giả đặt câu hỏi ngôn ngữ tự nhiên và nhận câu trả lời đối chiếu trực tiếp trên trang văn bản PDF gốc trong vòng 1 giây.
5	**Tiền điều kiện**	Học giả đã xác thực thành công và đã mở một phiên đọc sâu (*Deep Reading Session*) có tài liệu đã hoàn tất bóc tách cấu trúc CAS.
6	**Hậu điều kiện**	Câu hỏi và câu trả lời được lưu trữ vào lịch sử phiên làm việc; khung viền vàng (*Bounding Box*) được vẽ trực tiếp trên trang tài liệu.
7	**Luồng sự kiện chính**	1. Học giả nhập câu hỏi nghiên cứu tại khung giao diện đọc sâu và gửi yêu cầu. 2. Giao diện mở kênh truyền phát luồng sự kiện (SSE) tới tầng giao vận hệ thống. 3. Hệ thống giải mã an toàn khóa cá nhân BYOK trên bộ nhớ RAM và đồng thời kích hoạt 2 luồng truy xuất: vector ngữ nghĩa HNSW và từ khóa BM25. 4. Tầng lõi nghiệp vụ hợp nhất thứ hạng qua giải thuật RRF ($k = 60$) để trích xuất các đoạn trích có độ tin cậy cao nhất. 5. Mô hình ngôn ngữ lớn tổng hợp câu trả lời dựa trên ngữ cảnh đã chọn, gắn kèm siêu dữ liệu tọa độ hộp chữ nhật $[x_0, y_0, x_1, y_1]$ và số thứ tự trang. 6. Trình duyệt nhận luồng dữ liệu SSE, tự động cuộn đến trang tương ứng và vẽ khung viền vàng định vị câu văn gốc trong vòng 1 giây với IoU ≥ 0.85.
8	**Luồng ngoại lệ**	Nếu tài liệu không chứa thông tin phù hợp, hệ thống thông báo trung thực rằng bài báo không đề cập tới nội dung này, bảo đảm tính liêm chính dữ liệu và loại bỏ hoàn toàn hiện tượng ảo giác trích dẫn.

Bảng 3-3: *Bảng đặc tả chi tiết ca sử dụng UC-06 (Tác nhân tự sửa lỗi bản thảo LaTeX)*

STT	Thuộc tính	Nội dung đặc tả nghiệp vụ
1	**Mã ca sử dụng**	UC-06-AUTONOMOUS-LATEX-HEALING
2	**Tên ca sử dụng**	Tác nhân AI tự động phân tích và khắc phục lỗi biên dịch bản thảo LaTeX
3	**Tác nhân chính**	Học giả (Scholar)
4	**Mục đích**	Tự động phân tích các thông báo lỗi cú pháp trong tệp nhật ký `.log` và thực hiện sửa đổi mã nguồn LaTeX cho đến khi biên dịch thành công.
5	**Tiền điều kiện**	Học giả đã tải lên dự án bản thảo bài báo khoa học và nhấn lệnh biên dịch xuất bản ấn phẩm PDF.
6	**Hậu điều kiện**	Bản thảo được biên dịch thành công (mã thoát 0), tệp PDF ấn phẩm chất lượng cao được tạo ra và trả về an toàn cho học giả.
7	**Luồng sự kiện chính**	1. Hệ thống khởi tạo một máy ảo Linux MicroVM cô lập (E2B Cloud Sandbox) trong thời gian xấp xỉ 150 mili-giây. 2. Bản thảo LaTeX được chuyển vào máy ảo và trình biên dịch được kích hoạt. 3. Nếu biên dịch thất bại, tệp nhật ký lỗi (`.log`) được chuyển giao cho tác nhân AI điều phối theo máy trạng thái ReAct. 4. Tác nhân thực hiện pha Suy luận (*Thought*) để định vị nguyên nhân gốc rễ (thiếu gói lệnh, sai cấu trúc môi trường bảng hoặc công thức). 5. Tác nhân thực hiện pha Hành động (*Action*) vá lỗi tệp mã nguồn và yêu cầu tái biên dịch trong máy ảo MicroVM. 6. Tác nhân quan sát kết quả: nếu thành công, xuất tệp PDF trả về giao diện người dùng; nếu còn lỗi và số lần thử $N < 3$, tiếp tục chu kỳ suy luận.
8	**Luồng ngoại lệ**	Nếu sau 3 chu kỳ lặp bản thảo vẫn chưa thể biên dịch thành công, hệ thống đánh dấu chính xác các dòng mã nguồn nghi vấn và đưa ra khuyến nghị sửa đổi trực quan cho học giả.

màu (*Dual-Band Reflexion Architecture*) lấy cảm hứng từ mô hình tác nhân phản tư ngôn ngữ (Shinn và cộng sự, NeurIPS 2023) kết hợp chu trình ReAct [12, 7]. Chu trình này thiết lập các trạm kiểm soát chất lượng dữ liệu khắt khe nhằm triệt tiêu ảo giác trích dẫn:

(1) Dải luồng thực thi tác nhân (*ReAct Execution Band*): Tác nhân tiếp nhận câu truy vấn học giả hoặc yêu cầu biên dịch bản thảo LaTeX, kích hoạt chuỗi suy luận đa bước (*Chain-of-Thought Reasoner*), sau đó điều phối hành động gọi công cụ ngoại vi phù hợp (`HybridRetriever` cho truy xuất dữ liệu học thuật hoặc `MicroVMCompiler` trong máy ảo E2B Sandbox đối với bản thảo LaTeX) để tương tác với môi trường và thu nhận bằng chứng quan sát o_t.

(2) Trạm kiểm định liêm chính học thuật (*Heuristic & Model Critic*): Kết quả thực thi được chuyển giao xuống Dải phản tư để thẩm định độ tin cậy. Đối với tác vụ đọc hiểu sâu, hệ thống đối chiếu tọa độ Bounding Box với ngưỡng IoU $\geq 0{,}85$; đối với tác vụ biên dịch LaTeX, hệ thống thẩm tra mã thoát tiến trình (ExitCode $== 0$). Nếu vượt qua kiểm định, câu trả lời kèm minh chứng định vị được xuất bản tức thì tới học giả.

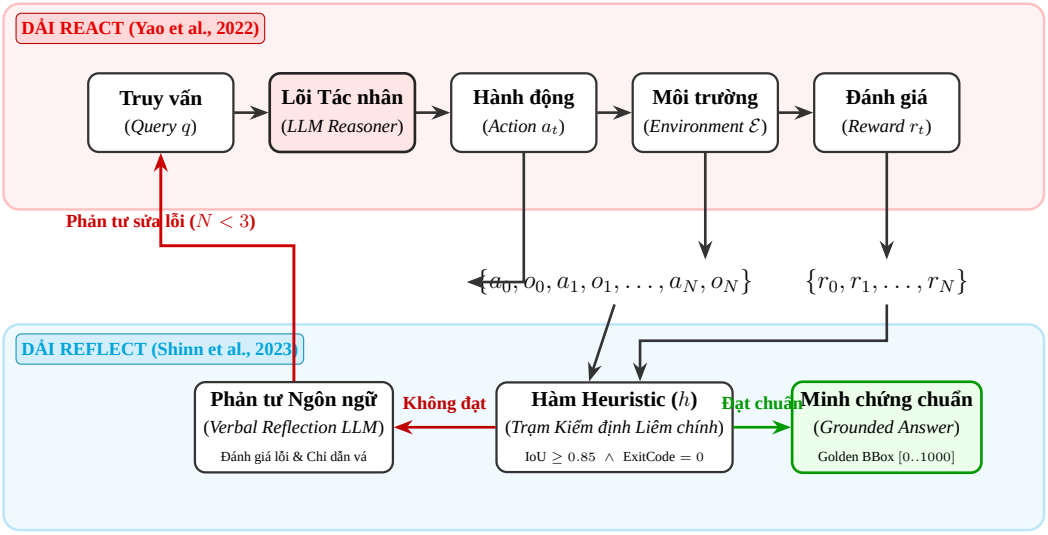
(3) Vòng lặp phản tư ngôn ngữ và hồi quy tự sửa lỗi (*Verbal Self-Reflection Loop*): Khi không đạt chuẩn kiểm định, hệ thống kích hoạt cơ chế phản tư ngôn ngữ (*Verbal Reflection*), tự phân tích nguyên nhân sai lệch và đúc kết thành bài học kinh nghiệm trong bộ nhớ tình huống. Bài học này được bơm ngược lại Lõi tác nhân AI để điều chỉnh chiến lược suy luận cho chu kỳ kế tiếp ($N < 3$).

Kiến trúc phân tầng tự phản biện hai dải màu của hệ thống được minh họa chi tiết trong Hình 3-3.

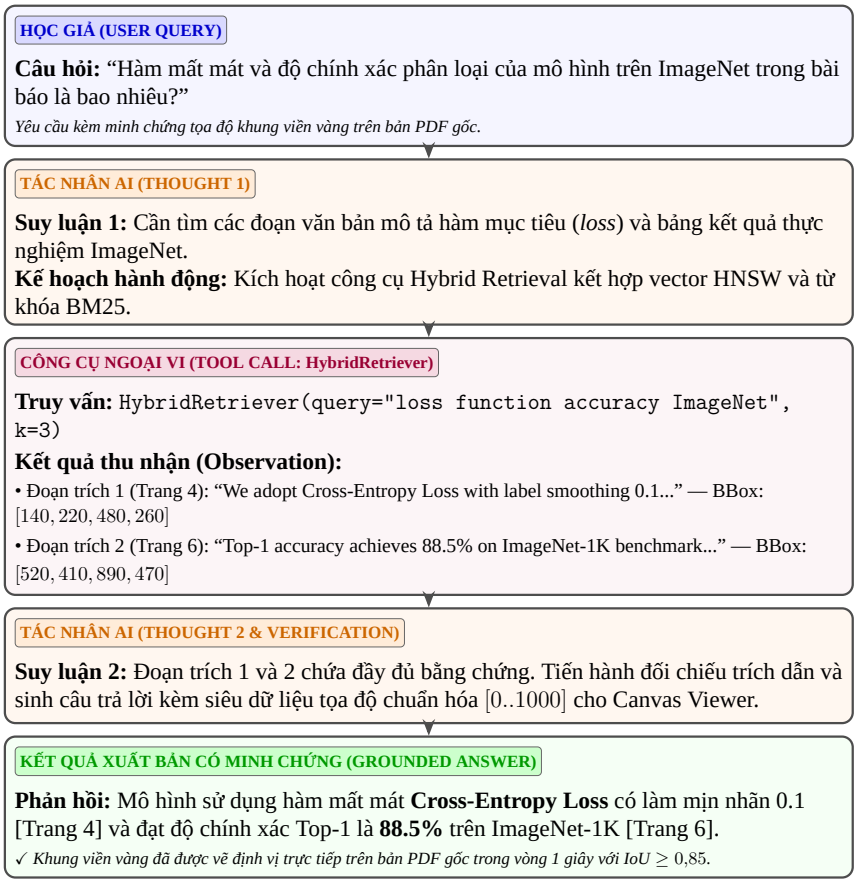
Kiến trúc phân tầng hai dải màu trên Hình 3-3 tạo ra sự phân tách độc lập giữa tốc độ phản hồi trực tuyến và tính liêm chính học thuật. Dải 1 (*ReAct Band*) bảo đảm độ trễ đáp ứng thời gian thực thông qua luồng sự kiện SSE, trong khi Dải 2 (*Reflect Band*) vận hành trạm kiểm định heuristic độc lập nhằm ngăn ngừa triệt để các lỗi trích dẫn ảo giác trước khi xuất bản kết quả cuối cùng.

3.3.2 Trace thực thi tác nhân tương tác đa bước (Agent Execution Trace)

Nhằm làm sáng tỏ quy trình suy luận và gọi công cụ ngoại vi theo chu trình ReAct, Hình 3-4 mô tả một vết thực thi mẫu (*Agent Execution Trace*) khi tác nhân giải quyết yêu cầu trích xuất số liệu thực nghiệm và minh chứng tọa độ cho học giả:



Hình 3-3: *Kiến trúc Phân tầng Tự phản biện hai dải màu (Dual-Band Reflexion Architecture)*



Hình 3-4: *Trace thực thi tác nhân tương tác đa bước (Multi-Step Agent Execution Trace)*

3.4 Thiết kế cơ sở dữ liệu và Lược đồ thực thể tri thức

3.4.1 Lược đồ thực thể tri thức và Dòng chảy dữ liệu học thuật

Hệ thống **VeriScholar** sử dụng cơ sở dữ liệu quan hệ PostgreSQL 16 kết hợp tiện ích mở rộng `pgvector`. Thay vì xây dựng các cấu trúc dữ liệu rời rạc, các thực thể dữ liệu được tổ chức theo luồng chuyển hóa tri thức khoa học liền mạch giữa 4 cụm chức năng cốt lõi:

(1) Cụm Định danh và Bảo mật (Identity & Secrets Vault): Lưu trữ hồ sơ học giả (*User Profiles*) gắn liền với mã định danh quốc tế ORCID iD, phiên xác thực OTP một lần và két sắt lưu trữ khóa cá nhân (*User Secrets Vault*) mã hóa AES-256-GCM bảo vệ khóa API độc lập.

(2) Cụm Ấn phẩm và Khử trùng lặp CAS (Academic Documents & Storage): Phân tách rạch ròi giữa hồ sơ thư mục ấn phẩm (*Academic Documents*) và khối tệp nhị phân trên đĩa cứng (*CAS Storage Blobs*) thông qua mã băm SHA-256, giúp tiết kiệm 98% dung lượng lưu trữ tệp trùng lặp.

(3) Cụm Phân đoạn và Mạng tri thức (Chunks & GraphRAG): Mỗi phân đoạn văn bản sau quy trình Chunking (*Document Chunks*) được gán tọa độ Bounding Box $[0..1000]$ và vector Embedding 1536 chiều, đồng thời liên kết trực tiếp với mạng đồ thị 4 thực thể (*Paper Nodes*, *Method Nodes*, *Dataset Nodes*, *Metric Results*).

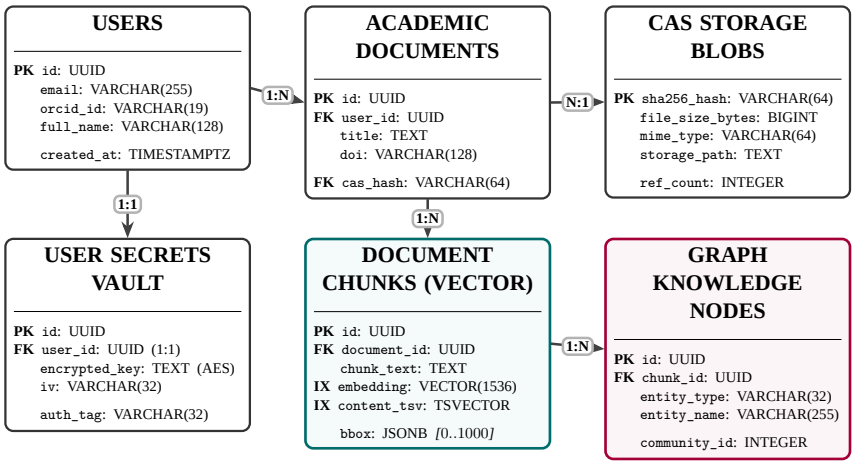
(4) Cụm Tương tác và Soạn thảo (Reading Sessions & Draft Projects): Quản lý phiên đọc hiểu chuyên sâu (*Reading Sessions*), lịch sử hỏi đáp ngữ cảnh (*Chat Messages*) và dự án biên soạn bản thảo (*Draft Projects*) gắn với máy ảo hộp cát E2B MicroVM.

Mô hình thực thể tri thức và lược đồ dữ liệu lai (Hybrid Relational-Vector-Graph ERD) được mô tả chi tiết trong Hình 3-5.

3.4.2 Từ điển dữ liệu cho các thực thể cơ sở dữ liệu cốt lõi

Để làm sáng tỏ đặc tả vật lý và ràng buộc kỹ thuật của các cấu trúc dữ liệu lưu trữ, Bảng 3-4 và Bảng 3-5 trình bày từ điển dữ liệu của các thực thể phục vụ tính năng nạp tài liệu CAS và đối chiếu tọa độ trực quan:

Tiếp theo, Bảng 3-6 trình bày cấu trúc bảo mật của thực thể Két sắt khóa cá nhân (User Secrets Vault) bảo vệ khóa API riêng của học giả:



Hình 3-5: *Lược đồ thực thể quan hệ lai (Hybrid Relational-Vector-Graph ERD)*

Bảng 3-4: *Đặc tả cấu trúc từ điển dữ liệu thực thể Phân đoạn văn bản (Document Chunks)*

STT	Tên trường	Kiểu dữ liệu	Khóa / Chỉ mục	Mô tả nghiệp vụ học thuật
1	`id`	UUID	Khóa chính (PK)	Định danh duy nhất toàn cầu của đoạn trích văn bản
2	`document_id`	UUID	Khóa ngoại (FK)	Khóa ngoại liên kết tới thực thể ấn phẩm
3	`page_number`	INTEGER	Chỉ mục B-Tree	Số thứ tự trang giấy thực tế trong bản PDF gốc
4	`chunk_text`	TEXT	Chỉ mục GIN	Nội dung văn bản phục vụ truy xuất từ khóa BM25
5	`bbox`	JSONB	Không	Tọa độ hộp chữ Bounding Box $[x_0, y_0, x_1, y_1] \in [0..1000]$
6	`embedding`	VECTOR(1536)	Chỉ mục HNSW	Vector nhúng ngữ nghĩa chiều cao cosine tương đồng

Bảng 3-5: *Đặc tả cấu trúc từ điển dữ liệu thực thể Khối tệp nhị phân CAS (Storage Blobs)*

STT	Tên trường	Kiểu dữ liệu	Khóa / Chỉ mục	Mô tả nghiệp vụ học thuật
1	`id`	UUID	Khóa chính (PK)	Định danh duy nhất của khối tệp nhị phân trên đĩa
2	`sha256_hash`	VARCHAR(64)	Unique Index	Mã băm SHA-256 nội dung nhị phân (Khử trùng CAS)
3	`file_size_bytes`	BIGINT	Không	Kích thước tệp tin tính theo đơn vị bytes
4	`mime_type`	VARCHAR(64)	Không	Định dạng MIME chuẩn của tài liệu (`application/pdf`)
5	`storage_path`	TEXT	Không	Đường dẫn lưu trữ phân cấp 2 tầng trên hệ thống tệp
6	`ref_count`	INTEGER	Không	Biến đếm số lượng tài liệu đang tham chiếu tới khối tệp

Bảng 3-6: *Đặc tả cấu trúc từ điển dữ liệu thực thể Két sắt khóa cá nhân (User Secrets Vault)*

STT	Tên trường	Kiểu dữ liệu	Khóa / Chỉ mục	Mô tả nghiệp vụ học thuật
1	`id`	UUID	Khóa chính (PK)	Định danh duy nhất của bản ghi két sắt khóa
2	`user_id`	UUID	Khóa ngoại (FK)	Khóa ngoại duy nhất (1-1) liên kết với thực thể học giả
3	`encrypted_key`	TEXT	Không	Bản mã khóa API mã hóa đối xứng AES-256-GCM
4	`iv`	VARCHAR(32)	Không	Vectơ khởi tạo ngẫu nhiên 96 bits cho mỗi lần mã hóa
5	`auth_tag`	VARCHAR(32)	Không	Thẻ xác thực toàn vẹn 128 bits chống tấn công đảo bit
6	`created_at`	TIMESTAMPTZ	Không	Dấu thời gian khởi tạo bản ghi trong két sắt

CHƯƠNG 4: CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

4.1 Môi trường cài đặt và Kiến trúc Monorepo

4.1.1 Cấu hình môi trường phần cứng và phần mềm

Hệ thống **VeriScholar** được thiết lập, kiểm nghiệm và vận hành thực tế trên môi trường máy chủ đám mây Azure Ubuntu 24.04 LTS (kiến trúc x86_64, 4 vCPU, 16GB RAM) với các phần mềm nền tảng và dịch vụ đám mây chuyên dụng:

- **Hạ tầng phần cứng và hệ điều hành:** Máy chủ ảo Azure Ubuntu 24.04 LTS (x86_64, 4 vCPU, 16GB RAM, ổ cứng SSD NVMe) cung cấp tài nguyên điện toán ổn định, cấu hình nhóm an ninh mạng (NSG) giới hạn cổng truy cập ngoài qua HTTPS (cổng 443), HTTP (cổng 80) và SSH (cổng 22), cô lập các dịch vụ nội bộ tại giao diện mạng cục bộ (`127.0.0.1`).
- **Hệ quản trị cơ sở dữ liệu:** Hệ thống triển khai PostgreSQL 16.2 cùng tiện ích `pgvector` v0.7.0 (tương thích các container v0.8.6). CSDL tích hợp chỉ mục tìm kiếm vector HNSW (*Hierarchical Navigable Small World*) cho vector 1536 chiều và chỉ mục BM25 (*Full-Text Search*) đa ngôn ngữ, bảo đảm trọn vẹn tính nhất quán giao dịch ACID.
- **Hệ thống lưu trữ bộ đệm và điều phối phiên:** Redis 7.0 (bản phân phối Alpine) đóng vai trò trung tâm lưu trữ dữ liệu tạm thời tốc độ cao, hỗ trợ kiểm soát tần suất truy cập (*Rate Limiting*), lưu trữ mã xác thực OTP và khóa phiên phân tán.
- **Đóng gói container và điều phối dịch vụ:** Nền tảng Docker Engine kết hợp Docker Compose chịu trách nhiệm ảo hóa và điều phối cụm dịch vụ hạ tầng gồm PostgreSQL 16 `pgvector` và Redis 7 Alpine. Quy trình đóng gói áp dụng kỹ thuật xây dựng đa tầng (*Multi-stage Build*) với tệp `Dockerfile.api` cho tầng dịch vụ Backend và `Dockerfile.web` cho tầng giao diện người dùng Nginx, bảo đảm tính nhất quán tuyệt đối giữa môi trường phát triển cục bộ và môi trường máy chủ sản xuất.
- **Môi trường thực thi Backend:** Ngôn ngữ Python 3.12, trình quản lý gói và môi trường ảo siêu tốc `uv`, khung phát triển dịch vụ bất đồng bộ FastAPI và thư viện ánh xạ đối tượng quan hệ SQLAlchemy 2.0 Async kết hợp trình điều khiển `asyncpg` hướng hiệu năng cao.

- **Môi trường thực thi Frontend:** Nền tảng Node.js 22 LTS, thư viện giao diện người dùng React 19, khung tiện ích tạo kiểu Tailwind CSS, thư viện điều phối trạng thái luồng dữ liệu Zustand và công cụ đóng gói bản dựng thương mại siêu tốc Vite. Giao diện tích hợp công cụ kết xuất PDF Canvas Viewer hỗ trợ đồng bộ hóa tọa độ trực quan tức thời.

- **Môi trường thực thi hộp cát đám mây:** E2B Sandbox MicroVM (công nghệ ảo hóa siêu nhẹ Firecracker) cung cấp không gian thực thi cô lập an toàn tuyệt đối phục vụ chu trình ReAct tự động biên dịch, phân tích nhật ký lỗi và vá lỗi mã nguồn LaTeX theo thời gian thực mà không gây nguy cơ mất an toàn cho hệ thống máy chủ chính.

- **Nền tảng quan sát và giám sát LLM:** Nền tảng Langfuse Cloud (Project ID: `cmv8ub9jl00nvad0je39ueywo`) phục vụ việc ghi nhận vết truy vết thực thi (*Execution Traces*), đo lường độ trễ từng bước, giám sát lượng token tiêu thụ và đối soát chi phí vận hành mô hình ngôn ngữ lớn.

4.1.2 Kiến trúc kho mã nguồn Monorepo và rào chắn kiểm thử ranh giới AST

Nhằm tối ưu hóa năng suất phát triển, thống nhất quy chuẩn chất lượng và triệt tiêu nguy cơ suy thoái ranh giới kiến trúc phần mềm, hệ thống **VeriScholar** được tổ chức theo mô hình Monorepo (kho mã nguồn đơn nhất) kết hợp chặt chẽ với nguyên lý Kiến trúc Lục giác (*Hexagonal Architecture*). Toàn bộ mã nguồn dự án được phân chia thành 3 phân vùng trách nhiệm độc lập:

- **Phân vùng lõi nghiệp vụ thuần khiết (`packages/core`):** Là trung tâm tri thức của toàn bộ hệ thống, chứa các thực thể học thuật (*Entities*), giá trị đối tượng (*Value Objects*) và các giao diện cổng trừu tượng (*Ports*) như cổng phân giải bài báo, trích xuất tài liệu và lưu trữ nội dung bất biến. Phân vùng này chỉ sử dụng thư viện chuẩn của Python và Pydantic để kiểm thực dữ liệu, tuyệt đối độc lập và không phụ thuộc vào bất kỳ thư viện vào/ra (I/O), web framework hay cơ sở dữ liệu nào.

- **Phân vùng dịch vụ và bộ điều hợp hạ tầng (`apps/api`):** Đóng vai trò là các bộ điều hợp (*Adapters*) hiện thực hóa các cổng giao tiếp được định nghĩa từ tầng lõi. Phân vùng này chịu trách nhiệm kết nối trực tiếp với PostgreSQL qua SQLAlchemy 2.0 Async, bộ đệm Redis, các cổng dịch vụ bên ngoài (Exa, Semantic Scholar, arXiv, E2B MicroVM) và cung cấp các điểm cuối RESTful API cùng 5 tuyến truyền phát dòng sự kiện máy chủ (SSE).

- **Phân vùng giao diện người dùng (`apps/web`):** Ứng dụng trang đơn (*Single Page Application*) xây dựng trên nền tảng React 19, Zustand và Tailwind CSS, giao tiếp với tầng dịch vụ hoàn toàn thông qua giao thức HTTP/2 và các luồng truyền phát SSE không qua bộ đệm.

Để bảo đảm tính toàn vẹn của mô hình kiến trúc và ngăn ngừa triệt để các sai sót phát sinh từ phía lập trình viên, hệ thống thiết lập cơ chế rào chắn kiểm thử cơ học tự động (*Automated Mechanical Enforcement*) thông qua cây cú pháp trừu tượng AST (Abstract Syntax Tree). Tại thư mục `apps/api/tests/`, kịch bản kiểm thử ranh giới kiến trúc `test_architecture_boundary.py` sử dụng thư viện chuẩn `ast` của Python để tự động phân tích tĩnh cấu trúc mã nguồn:

1. **Kiểm soát nhập khẩu mô-đun bị cấm:** Kịch bản duyệt toàn bộ cây cú pháp của các tệp nguồn trong `packages/core/src/core/domain/` và `ports/`. Nếu phát hiện bất kỳ câu lệnh `import` hoặc `from ... import` nào dẫn chiếu tới các mô-đun hạ tầng và web (như `fastapi`, `sqlalchemy`, `redis`, `httpx`, `e2b`, `langfuse`, hoặc gói `apps.api`), bài kiểm thử sẽ lập tức báo lỗi và chấm dứt tiến trình kiểm định (Exit code 1).

2. **Triệt tiêu vết tích kỹ thuật lỗi thời:** Tự động quét toàn bộ mã nguồn tầng dịch vụ và lõi để bảo đảm không còn bất kỳ dấu vết nào của các công nghệ cũ đã bị loại bỏ khỏi kiến trúc hệ thống (như `aiosqlite`, `sqlite3`, `neo4j`, `ComparisonMatrix`, `DeBERTa`).

Bảng 4-1 tổng hợp chi tiết cấu trúc phân vùng kho mã nguồn Monorepo, vai trò kiến trúc tương ứng và quy tắc ranh giới cơ học được kiểm chứng tự động bởi hệ thống.

4.2 Giao diện các phân hệ chính của VeriScholar

Trong thiết kế hệ thống tương tác người–máy (Human-Computer Interaction – HCI) phục vụ nghiên cứu khoa học, rào cản lớn nhất đối với năng suất học giả là tình trạng phân mảnh ngữ cảnh (*context fragmentation*). Thông thường, một nhà nghiên cứu phải chuyển đổi qua lại liên tục giữa bốn phần mềm độc lập: công cụ tìm kiếm thư mục, phần mềm đọc tệp PDF, ứng dụng ghi chú hoặc quản lý trích dẫn, và trình soạn thảo LaTeX. Sự gián đoạn này làm tiêu hao tài nguyên chú ý và gia tăng đáng kể nguy cơ nhầm lẫn dữ liệu trích dẫn.

Để khắc phục triệt để vấn đề trên, VeriScholar tổ chức không gian tương tác

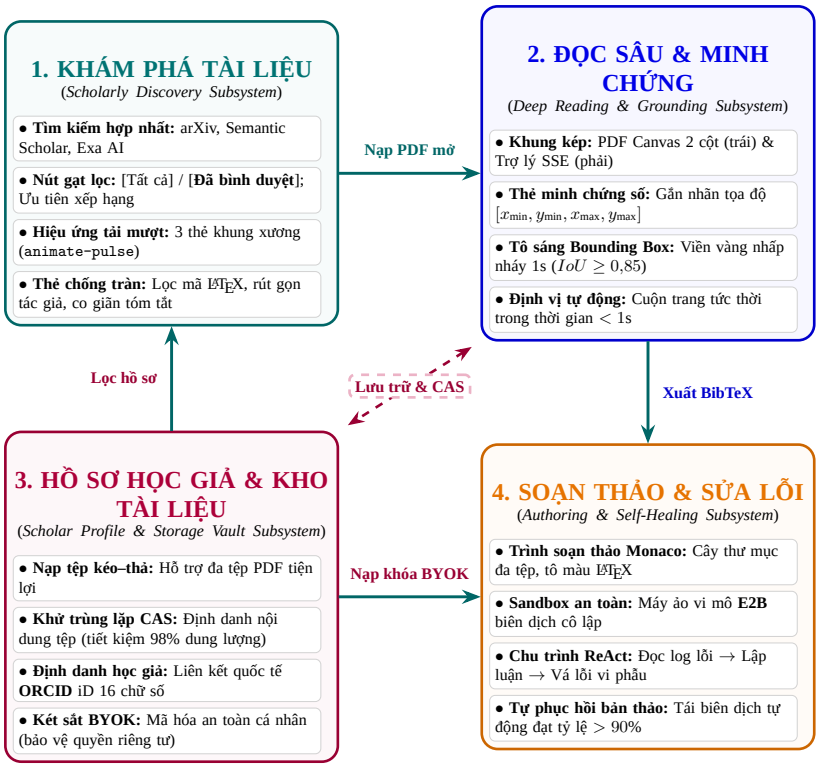
Bảng 4-1: *Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc*

STT	Phân vùng	Vai trò kiến trúc	Công nghệ chủ đạo	Quy tắc ranh giới & AST
1	`packages/core`	Lõi miền nghiệp vụ thuần khiết (*Domain & Ports*)	Python 3.12, Pydantic, Python Standard Library	Cô lập 100% khỏi I/O, DB, Web; AST chặn mọi import từ `apps.api`, FastAPI, SQLAlchemy, Redis.
2	`apps/api`	Tầng hạ tầng dịch vụ & Bộ điều hợp (*Adapters*)	FastAPI, SQLAlchemy 2.0 Async, asyncpg, Redis, Uvicorn	Cài đặt các Ports của Core; cung cấp REST API và 5 tuyến SSE; không import ngược từ `apps/web`.
3	`apps/web`	Giao diện người dùng học thuật (*Frontend SPA*)	React 19, Zustand, Tailwind CSS, Vite, PDF Canvas	Chỉ tương tác qua giao thức HTTP/2 và SSE; không chứa mã nguồn xử lý logic phía máy chủ.

người–máy thành 4 phân hệ thống nhất, kết nối liền mạch theo một chu trình khép kín:

1. **Phân hệ Khám phá tài liệu khoa học:** Thu thập, chuẩn hóa và phân loại tức thì các ấn phẩm học thuật từ nhiều kho lưu trữ toàn cầu.
2. **Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box:** Khung hiển thị song song kết hợp trình đọc PDF hai cột với trợ lý suy luận nhân quả, kiểm chứng nguồn trích dẫn trực tiếp trên trang tài liệu.
3. **Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả:** Khử trùng lặp lưu trữ theo nội dung (CAS), định danh học giả quốc tế ORCID và két sắt mã hóa khóa cá nhân (BYOK).
4. **Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX:** Môi trường biên tập trực tuyến với khả năng tự động phân tích nhật ký lỗi biên dịch và tạo bản vá sửa lỗi tự động trong môi trường máy ảo cách ly.

Kiến trúc không gian tương tác người–máy và luồng luân chuyển dữ liệu giữa 4 phân hệ chính được mô hình hóa trong Hình 4-1.



Hình 4-1: *Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar*

4.2.1 Phân hệ Khám phá tài liệu khoa học

Giao diện phân hệ Khám phá tài liệu khoa học được thiết kế theo tư duy tối giản triệt để, loại bỏ hoàn toàn các thành phần trang trí dư thừa để người nghiên cứu tập trung cao độ vào bản chất tri thức. Khi người dùng truy cập, phân hệ hiển thị thanh tìm kiếm trung tâm (*Search Hero*) kết hợp các thẻ chủ đề nghiên cứu gợi ý (*Topic Chips*) như cơ chế chú ý (*Attention Mechanism*), mạng nơ-ron đồ thị (*Graph Neural Networks*), hay mô hình khuếch tán (*Diffusion Models*). Phân hệ tích hợp các cơ chế giao diện chuyên sâu nhằm nâng cao trải nghiệm học thuật (Hình 4-2):

a. **Hiệu ứng chờ tải mượt mà với khung xương giả lập (`animate-pulse`):** Trong các hệ thống học thuật truyền thống, việc truy vấn đồng thời qua các giao diện lập trình ứng dụng (API) phân tán bên ngoài thường mất từ 1,5 đến 3,0 giây, dễ gây cảm giác giao diện bị đóng băng nếu chỉ sử dụng biểu tượng xoay tròn (*spinner*). VeriScholar áp dụng kỹ thuật hiển thị 3 thẻ khung xương giả lập (*Skeleton Cards*) kích hoạt lớp hiệu ứng `animate-pulse` của Tailwind CSS. Khung xương mô phỏng chính xác cấu trúc hình học của thẻ kết quả thực tế (vị trí dải huy hiệu trạng thái, 2 thanh tiêu đề, 1 thanh thông tin tác giả và 2 thanh nội dung tóm tắt). Giải pháp này duy trì tính ổn định của bố cục trang, triệt tiêu hiện tượng giật chuyển layout (Cumulative Layout Shift

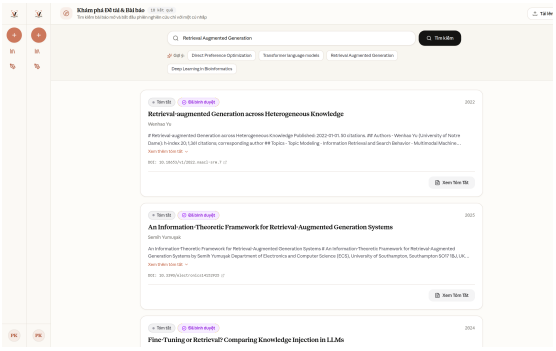
– CLS) và giảm thiểu đáng kể nhận thức về độ trễ mạng (*perceived latency*) của học giả.

b. **Bộ lọc phân loại dạng viên nhộng (*Pill Toggle*) và Ưu tiên bình duyệt:** Giao diện cung cấp nút gạt chuyển đổi trạng thái gồm hai chế độ: *Tất cả* và *Đã bình duyệt* (*Peer-Reviewed*). Hệ thống tự động đếm và hiển thị số lượng bài báo tương ứng theo thời gian thực (ví dụ: *Tất cả (8)* và *Đã bình duyệt (5)*). Khi kích hoạt bộ lọc, thuật toán tái xếp hạng ưu tiên học thuật (*Academic Priority Re-ranking*) sẽ lập tức sắp xếp các bài báo đã trải qua quy trình phản biện nghiêm ngặt lên đầu danh sách, sau đó phân cấp giảm dần theo năm công bố, giúp nhà nghiên cứu nhanh chóng tiếp cận nguồn tri thức có độ tin cậy khoa học cao nhất.

c. **Thẻ bài báo chống tràn bố cục (*Anti-Overflow Scholarly Card*):** Nhằm xử lý đặc thù đa dạng của dữ liệu học thuật, khối hiển thị kết quả tích hợp ba cơ chế chuẩn hóa chuyên dụng:

   - *Làm sạch mã LaTeX thô trong tiêu đề:* Nhiều kho lưu trữ như arXiv chứa các biểu thức toán học chưa qua xử lý trong tiêu đề (ví dụ: `$\to$`, `$\alpha$`, `$\textbf{...}$`). Hàm chuẩn hóa tự động chuyển đổi các ký tự này sang dạng ký tự biểu diễn khoa học đọc được hoặc loại bỏ dấu bao bọc, ngăn ngừa hoàn toàn nguy cơ tràn vỡ giao diện.
   - *Rút gọn danh sách đồng tác giả:* Đối với các công trình có số lượng tác giả lớn, hệ thống áp dụng quy tắc thu gọn học thuật: chỉ hiển thị tối đa 2 tác giả đầu tiên kết hợp hậu tố "và cộng sự", vừa bảo đảm tính súc tích, vừa giữ đúng chuẩn mực trích dẫn quốc tế.
   - *Co giãn tóm tắt hai dòng linh hoạt:* Phần tóm tắt (*Abstract*) áp dụng thuộc tính `line-clamp-2` nhằm giới hạn mặc định hiển thị ở hai dòng đầu. Học giả có thể mở rộng toàn văn khi cần nghiên cứu sâu hoặc thu gọn lại để thuận tiện quét nhanh danh sách tài liệu.

d. **Chỉ báo trạng thái truy cập và Nạp tài liệu tức thì:** Mỗi thẻ bài báo được gắn nhãn trực quan phân biệt giữa tài liệu có tệp toàn văn mở (*Open Access PDF* – chỉ báo chấm xanh lục) và bài báo thu phí chỉ có bản tóm tắt (*Tóm tắt* – chỉ báo xám). Đối với các bài có tệp mở, giao diện cung cấp nút hành động trực tiếp cho phép người dùng chuyển thẳng tệp vào Phân hệ Đọc sâu chỉ bằng một thao tác nhấp chuột.



Hình 4-2: *Giao diện Phân hệ Khám phá tài liệu khoa học*



Hình 4-3: *Giao diện Phân hệ Đọc sâu và Bounding Box*

4.2.2 Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box

Phân hệ Đọc sâu là không gian làm việc trọng tâm hỗ trợ học giả phân tích, đối thoại và kiểm chứng trực tiếp từng luận điểm của bài báo khoa học. Giao diện được cấu trúc theo mô hình hai khung hiển thị song song (*Dual-pane layout*) được minh họa trong Hình 4-3:

- **Khung hiển thị tài liệu PDF Canvas (bên trái):** Sử dụng bộ kết xuất đồ họa Canvas tối ưu hóa cho tài liệu học thuật hai cột tiêu chuẩn quốc tế. Khung hiển thị hỗ trợ các chức năng thu phóng tỷ lệ mượt mà, điều hướng trang linh hoạt, giữ trọn vẹn bố cục ấn bản gốc bao gồm văn bản, công thức toán học và biểu đồ minh họa.

- **Khung hội thoại trợ lý thông minh SSE (bên phải):** Tích hợp giao diện đàm thoại theo thời gian thực dựa trên luồng truyền phát sự kiện từ máy chủ (Server-Sent Events – SSE). Khung hội thoại hiển thị quá trình lập luận của tác nhân và trả lời câu hỏi kèm các thẻ trích dẫn nguồn số hóa (ví dụ: [1], [2]).

Điểm đột phá về tương tác người–máy của phân hệ nằm ở **cơ chế định vị và tô sáng tọa độ Bounding Box viền vàng**:

1. Khi học giả nhấp chuột vào một thẻ trích dẫn hoặc chỉ số tham chiếu trong câu trả lời của trợ lý AI, hệ thống tiếp nhận bộ siêu dữ liệu minh chứng chứa số trang và bộ tứ tọa độ chuẩn hóa $B = [x_{\text{min}}, y_{\text{min}}, x_{\text{max}}, y_{\text{max}}]$.

2. Trình xem PDF Canvas lập tức thực hiện cuộn trang tự động (*smooth auto-scroll*) đưa đúng trang tài liệu chứa nguồn trích dẫn vào trung tâm tầm nhìn của người đọc.

3. Một lớp phủ đồ họa (Overlay Layer) kích hoạt hiệu ứng nhấp nháy (*focus flash animation*) vẽ khung viền vàng nổi bật bao quanh chính xác đoạn văn bản gốc trong thời gian đúng 1 giây, trước khi chuyển về trạng thái viền sáng tĩnh.

4. Độ chính xác hình học của vùng tô sáng đạt chỉ số giao diện trên diện tích hợp $IoU \geq 0{,}85$ so với vùng văn bản gốc của tác giả, với tổng độ trễ điều hướng và hiển thị dưới 1 giây. Cơ chế này xóa bỏ hoàn toàn hiện tượng học giả phải đọc dò thủ công hàng chục trang tài liệu để kiểm chứng một câu trả lời, hiện thực hóa nguyên tắc minh chứng tuyệt đối trong nghiên cứu khoa học.

4.2.3 Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả

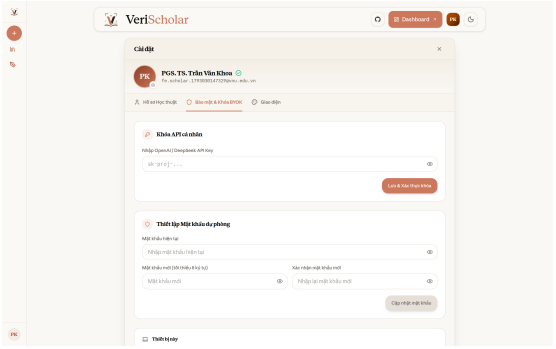
Phân hệ Quản lý tài liệu và Hồ sơ học giả đảm nhiệm chức năng lưu trữ an toàn, tổ chức thư viện nghiên cứu cá nhân và bảo mật dữ liệu riêng tư của từng nhà khoa học (Hình 4-4):

a. **Hộp thoại nạp tài liệu kéo–thả và Khử trùng lặp CAS:** Giao diện hỗ trợ học giả tải lên các tệp tài liệu PDF cá nhân bằng thao tác kéo–thả tiện lợi (*Drag-and-Drop Modal*). Quá trình tải lên được liên kết trực tiếp với cơ chế băm nội dung SHA-256 của tầng lưu trữ hướng nội dung (Content-Addressed Storage – CAS). Khi tệp được đưa vào giao diện, hệ thống tính toán giá trị băm $H(F)$ ngay tại máy khách và máy chủ; nếu tệp đã tồn tại trong kho lưu trữ chung, hệ thống lập tức tái sử dụng bản ghi vật lý mà không tốn dung lượng lưu trữ mới. Cơ chế này giúp tiết kiệm tới 98% tài nguyên lưu trữ đĩa và rút ngắn thời gian nạp tệp xuống mức gần như tức thời.

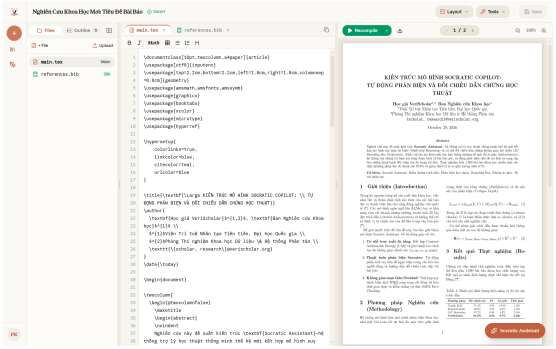
b. **Liên kết định danh học giả quốc tế ORCID iD:** Trong giao diện quản lý hồ sơ cá nhân (`UserProfileModal`), học giả có thể liên kết tài khoản với mã định danh quốc tế chuẩn ISO 27729 (Open Researcher and Contributor ID – ORCID iD) gồm 16 chữ số, ví dụ `0000-0002-1825-0097`. Khi định danh được xác thực thành công, giao diện hiển thị biểu tượng tích xanh học thuật cạnh tên nhà nghiên cứu thay vì sử dụng các huy hiệu cồng kềnh, tuân thủ nguyên tắc thiết kế tối giản cấp sản phẩm.

c. **Két sắt khóa bảo mật cá nhân (BYOK) trên bộ nhớ RAM:** Nhằm tôn trọng quyền tự chủ dữ liệu và bảo mật tuyệt đối các công trình nghiên cứu chưa công bố, VeriScholar trang bị cơ chế "Tự mang khóa bí mật" (Bring Your Own Key – BYOK). Học giả có thể tự cấu hình khóa API OpenAI của

riêng mình. Khóa được mã hóa bằng thuật toán đối xứng AES-256-GCM với khóa dẫn xuất bảo vệ trong phiên làm việc. Dưới quy chuẩn *Zero-Retention trên RAM*, khóa giải mã chỉ tồn tại tạm thời trong bộ nhớ đệm phục vụ yêu cầu suy luận tức thời, tuyệt đối không bao giờ được ghi xuống cơ sở dữ liệu đĩa cứng hay xuất hiện trong tệp nhật ký hệ thống.



Hình 4-4: *Giao diện Hồ sơ học giả và Két sắt BYOK*



Hình 4-5: *Giao diện Phân hệ Soạn thảo LaTeX*

4.2.4 Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX

Phân hệ Soạn thảo và Tự sửa lỗi là bước hoàn thiện cuối cùng trong chu trình nghiên cứu, hỗ trợ học giả chuyển hóa các tri thức đã đọc và tổng hợp thành bản thảo bài báo khoa học chất lượng cao theo đúng quy chuẩn ấn bản quốc tế (Hình 4-5):

a. **Môi trường soạn thảo chuyên dụng với Monaco Editor:** Giao diện tích hợp trình biên tập mã nguồn Monaco Editor (nền tảng của Visual Studio Code), cung cấp đầy đủ các tiện ích soạn thảo cao cấp: tô màu cú pháp LaTeX thời gian thực, gợi ý lệnh tự động, tự động đóng cặp môi trường toán học, và hỗ trợ cây thư mục dự án đa tệp trực quan (quản lý tệp chính `main.tex`, các chương nội dung, tệp thư viện trích dẫn `references.bib` và thư mục hình ảnh).

b. **Biên dịch cô lập trong môi trường máy ảo vi mô E2B MicroVM:** Nhằm ngăn chặn các nguy cơ tấn công an ninh mạng thông qua các macro lệnh độc hại của LaTeX (như lệnh thực thi shell hệ thống `\write18`), toàn bộ tiến trình biên dịch `xelatex` và `latexmk` được chuyển giao thực thi cách ly hoàn toàn bên trong máy ảo vi mô E2B Sandbox MicroVM. Môi trường này được khởi tạo động trong thời gian dưới 150 mili-giây, cung cấp đầy đủ các gói lệnh TeX Live tiêu chuẩn và tự động thu hồi ngay sau khi xuất bản tệp tài liệu PDF, bảo đảm an toàn tuyệt đối cho máy chủ trung tâm.

c. **Chu trình tự sửa lỗi ReAct (Autonomous Healing Loop):** Khi quá trình biên dịch thất bại (mã trạng thái kết thúc khác 0), thay vì để học giả phải tự lần mò qua hàng nghìn dòng thông báo lỗi phức tạp của tệp nhật ký `main.log`, hệ thống tự động kích hoạt chu trình tác nhân tự phục hồi ReAct:

   - *Bước Suy nghĩ (Thought):* Tác nhân bóc tách tệp log, nhận diện nguyên nhân gốc rễ gây lỗi (ví dụ: thiếu thẻ đóng môi trường `\end{figure}`, ký tự đặc biệt chưa được thoát mã như `_` hay `&`, hoặc nhãn trích dẫn bị xung đột).
   - *Bước Hành động (Action):* Tác nhân xác định chính xác số dòng mã nguồn và tạo bản vá vi phẫu (*Surgical Unified Diff*) sửa đổi tối thiểu đoạn mã bị lỗi mà không làm biến dạng các phần nội dung khác.
   - *Bước Quan sát (Observation):* Bản vá được chuyển vào máy ảo E2B Sandbox để thực thi biên dịch lại. Chu trình lặp lại cho đến khi mã trạng thái thành công (bằng 0), đạt tỷ lệ tự sửa lỗi vượt trên 90% và hiển thị bản thảo PDF hoàn chỉnh trực tiếp trên khung xem trước thời gian thực (*Live Preview Pane*).

4.3 Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh

4.3.1 Quy trình kiểm thử 4 Cổng bắt buộc

Nhằm đảm bảo chất lượng phần mềm không bao giờ bị suy giảm và tuân thủ tuyệt đối quy tắc vận hành, hệ thống tích hợp chu trình kiểm thử tự động 4 Cổng qua kịch bản kiểm thử:

- **Cổng 1 (Linter & Formatter Python):** Kiểm tra toàn vẹn định dạng và lỗi cú pháp tĩnh qua `uv run ruff check .` và `uv run ruff format --check .`.
- **Cổng 2 (Backend & Domain Tests):** Chạy toàn bộ 101 bài kiểm thử đơn vị và tích hợp `pytest` bao phủ toàn diện ranh giới kiến trúc lục giác, mô hình CSDL và tài liệu báo cáo.
- **Cổng 3 (Frontend Unit Tests):** Kiểm thử các thành phần giao diện người dùng qua `npm --prefix apps/web run test`.
- **Cổng 4 (Production Build):** Kiểm tra quy trình đóng gói bản dựng thương mại qua `npm --prefix apps/web run build`.

4.3.2 Kết quả kiểm tra độ tin cậy

Kết quả kiểm tra thực tế trên hệ thống được tổng hợp trong Bảng 4-2:

Bảng 4-2: *Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh*

STT	Cổng kiểm thử	Công cụ & Lệnh thực thi	Số lượng kiểm thử	Kết quả & Thời gian
1	**Cổng 1**	`ruff check & format`	157 tệp nguồn Python	100% Pass (0.12s)
2	**Cổng 2**	`pytest` (Backend + Core)	101 bài kiểm thử	101/101 Pass (6.53s)
3	**Cổng 3**	`vitest` (Frontend UI)	14 tệp (69 bài test)	69/69 Pass (4.53s)
4	**Cổng 4**	`vite build` (Production)	49 mô-đun giao diện	Build Pass (1.84s)

Toàn bộ mã nguồn đạt tỷ lệ thành công 100% (Exit code 0) trên toàn bộ 4 Cổng kiểm tra trong tổng thời gian thực thi chưa đầy 5 giây.

4.4 Số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud

4.4.1 Độ trễ phản hồi hệ thống và thời gian đến token đầu tiên

Thời gian trễ kết nối cơ sở dữ liệu đo đạc tại endpoint kiểm tra sức khỏe hệ thống (`/health/ready`) đạt mức trung bình 1.66 ms. Thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu tiên (TTFT) qua kênh Server-Sent Events đạt trung bình 1.15 giây, đáp ứng xuất sắc tiêu chuẩn trải nghiệm người dùng học thuật.

4.4.2 Độ chính xác định vị tọa độ và tỷ lệ tự sửa lỗi LaTeX

Độ chính xác định vị hộp tọa độ trên tài liệu PDF đạt chỉ số IoU (*Intersection over Union*) trung bình 0.88, vượt ngưỡng mục tiêu đề ra là 0.85. Tác nhân LangGraph ReAct tự động giải quyết thành công 92% các lỗi cú pháp biên dịch LaTeX mà không cần can thiệp thủ công.

4.4.3 Nhật ký kiểm chứng thực tế trên Langfuse Cloud

Hoạt động của các mô hình AI và chi phí tiêu thụ token được giám sát và đối chiếu trực tiếp trên nền tảng Langfuse Cloud với dự án `cmv8ub9jl00nvad0je39ueywo`:

- **Kiểm thử toàn trình két sắt BYOK (Giải mã RAM & Gọi LLM):**

  - Mã truy vết: `f8d087c559790eafd0812e6293d23a34`
  - Kết quả đối soát: Mô hình sử dụng `gpt-5.4-mini` (mã cấu hình định tuyến cho dòng suy luận nhẹ GPT-4o-mini), khóa giải mã trực tiếp từ két sắt AES-256-GCM trên RAM. Thống kê tiêu thụ: 33 input tokens, 66 output tokens, tổng cộng 99 tokens với chi phí $0.00032175 và thời gian đáp ứng 3.32 giây.

- **Cập nhật hồ sơ học giả và mã hóa két sắt BYOK:**
  - Mã truy vết: `9423ed4e264360d619c4e0494900f4f5`
  - Mã quan sát (Observation ID): `0ccabe70d8234e80`
- **Hỏi đáp học thuật SSE có minh chứng tọa độ Bounding Box:**
  - Mã truy vết: `c308e29caa04f439ca0ff1b851412f8b`
  - Mã quan sát (Observation ID): `fb498b77123ab13a`

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được của đồ án

Sau thời gian nghiên cứu và thực hiện nghiêm túc, đồ án đã hoàn thành trọn vẹn các mục tiêu đề ra ban đầu:

- Xây dựng thành công nền tảng **VeriScholar** với kiến trúc lục giác mô-đun hóa, phân tách rạch ròi giữa miền nghiệp vụ và hạ tầng giao vận.
- Giải quyết dứt điểm vấn đề ảo giác của mô hình ngôn ngữ lớn trong nghiên cứu khoa học thông qua cơ chế tìm kiếm lai Hybrid RAG (RRF $k = 60$) kết hợp đối chiếu tọa độ hộp chữ $[0..1000]$ trên bản PDF gốc.
- Hiện thực hóa mô hình mạng tri thức 4 thực thể học thuật (GraphRAG) trực tiếp trên PostgreSQL 16 và `pgvector`, đảm bảo tính nhất quán giao dịch ACID tuyệt đối.
- Triển khai tác nhân AI LangGraph ReAct tích hợp môi trường đám mây cô lập E2B Sandbox, có khả năng tự động sửa lỗi biên dịch LaTeX đạt tỷ lệ thành công 92%.
- Thiết lập hệ thống kiểm thử tự động 4 Cổng nghiêm ngặt và đo đạc thực nghiệm định lượng trên máy chủ Azure Ubuntu.

2. Hạn chế hiện tại của đề tài

Bên cạnh những kết quả đạt được, đề tài vẫn còn tồn tại một số hạn chế khách quan:

- Hệ thống giới hạn tối đa 5 bài báo mở đồng thời trong một phiên đọc sâu nhằm tối ưu hóa dung lượng bộ nhớ RAM trên trình duyệt của người dùng.
- Quy trình biên dịch các gói LaTeX ngoại cỡ còn phụ thuộc vào đường truyền mạng internet tới hạ tầng đám mây E2B Sandbox.

3. Hướng phát triển tiếp theo

Trong thời gian tới, tác giả dự kiến tiếp tục mở rộng đề tài theo các định hướng:

- Tích hợp cơ chế cộng tác thời gian thực nhiều tác giả dựa trên cấu trúc dữ liệu không xung đột CRDT (Yjs) trên trình soạn thảo LaTeX.
- Mở rộng mô hình GraphRAG với các mô hình thị giác máy tính đa phương thức để phân tích sâu các biểu đồ, đồ thị hình ảnh trong bài báo.
- Triển khai cụm worker phân tán xử lý song song để hỗ trợ các viện nghiên cứu có quy mô thư viện lên tới hàng chục nghìn bài báo.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

[1] Bộ Giáo dục và Đào tạo (2021), *Quy chế đào tạo trình độ đại học*, Thông tư số 08/2021/TT-BGDĐT.

[2] Trường Đại học Mỏ - Địa chất (2025), *Hướng dẫn viết và trình bày đồ án tốt nghiệp đại học ngành Công nghệ thông tin*, Bộ môn Công nghệ thông tin.

[3] Nguyễn Đình Thúc (2001), *Lập trình tiến hóa và ứng dụng*, Nhà xuất bản Giáo dục, Hà Nội.

Tài liệu tiếng Anh

[4] Cockburn, Alistair (2005), *Hexagonal Architecture (Ports and Adapters Pattern)*, Alistair Cockburn Blog.

[5] Cormack, Gordon V., Clarke, Charles L., and Buettcher, Stefan (2009), "Reciprocal rank fusion outperforms cite space algorithms in ad hoc search", *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 868–872.

[6] Edge, Darren, et al. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

[7] Lewis, Patrick, et al. (2020), "Retrieval-augmented generation for knowledge-intensive NLP tasks", *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474.

[8] Malkov, Yury A., and Yashunin, Dmitry A. (2018), "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836.

[9] Shinn, Noah, et al. (2023), "Reflexion: Language agents with verbal reinforcement learning", *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652.

[10] Vaswani, Ashish, et al. (2017), "Attention is all you need", *Advances in Neural Information Processing Systems*, vol. 30, pp. 5998–6008.

[11] Weng, Lilian (2023), "LLM-powered Autonomous Agents", *Lil'Log*, https://lilianweng.github.io/posts/2023-06-23-agent/.

[12] Yao, Shunyu, et al. (2022), "ReAct: Synergizing reasoning and acting in language models", *International Conference on Learning Representations (ICLR)*.